

ME350
Fall 2025 Semester
FINAL REPORT

Team 24

William Peper
Matthew Gordon
Evan Link
Joshua Benz
Elijah Iacovoni

GSI/IA: Hoaran Shi

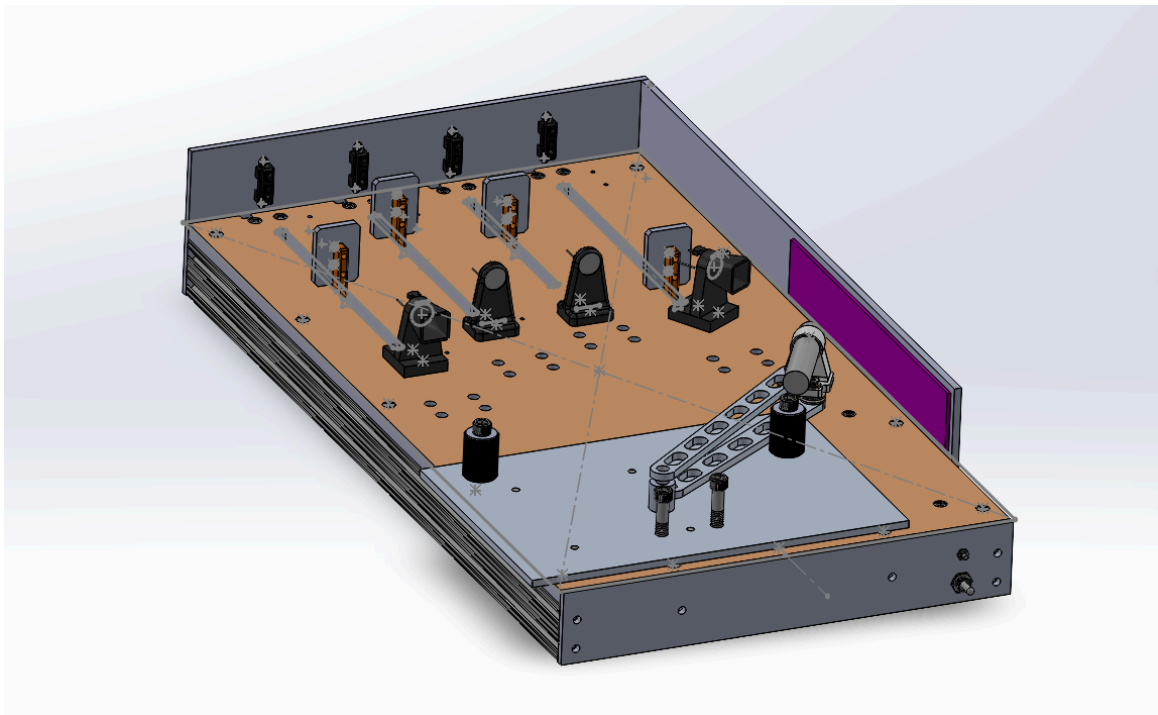


Figure 1.1: Linkage assembly on Playing Field

"We have fully abided by the University of Michigan College of Engineering Honor Code"

William Peper
Evan Link
Matthew Gordon
Josh Benz
Elijah Iacovani

Introduction

In this project, we investigated the design and analysis of a powered linkage mechanism intended to detect and respond to approaching targets, inspired by the *Plants vs. Zombies* design challenge. The system's objective was to position a flashlight with sufficient accuracy and speed to activate photosensors, thereby sending targets back before reaching a front limit switch. To achieve this, we developed a CAD model of the mechanism and analyzed its performance by calculating transmission angles to ensure optimal motion transfer. Using ADAMS, we then simulated the system's dynamic response to evaluate torque and power consumption under motor-driven actuation. This model-based approach provided insight into the relationship between linkage geometry, dynamic efficiency, and power requirements, guiding the selection of an effective and low-inertia mechanism for real-world implementation.

Section 1: Design Goals, Requirements, and Specifications

Goals

1. To create a linkage system capable of following the path created for it.
2. To have the output of the linkage system match the input values from the sensors.
3. Minimize number of missed targets to ensure higher score
4. Create a lightweight linkage system to avoid weight-related penalties

Requirements

1. The flashlight can light all targets
2. Linkage can move without stopping between each hard stop.
3. Input and follower links must be aluminum.
4. Must mount to pre-drilled holes on the playing field; no modifications to the playing field allowed.
5. Linkage should not bind.
6. Linkage should be able to support the weight of the flashlight without malfunctioning.
7. Less than 3 missed targets per round

Our first, most important requirement is for the flashlight to be able to light all targets, as this is the objective of the game. Since our design will be capable of lighting all of the targets, it should be capable of moving between each hard stop. We also want to focus on designs that allow us to maximize efficiency, mechanical advantage, and prevent any mechanical/technical failures. Other requirements are those posted in the guidelines for us to abide by, as mentioned in the 350 Project guide. As a team we decided that we wanted to minimize the power required by our final linkage which was used as a performance indicator.

Specifications:

1. Acceptable range for transmission angle (μ): $30^\circ \leq \mu \leq 150^\circ$

2. Maximum transmission angle deviation ($|90-\mu|$) must be less than 60°
3. Linkage system must be able to support a minimum load of $60g \pm 3g$
4. Input & follower linkages must each be less than 12" in length
5. Maximum power required must be $< 1W$

Most of our specifications revolve around transmission angle/deviation as we want to ensure that we don't encounter instances where our linkage system malfunctions or binds. If we fall outside of our ideal angle range, we lose efficiency & potential mechanical advantage from our designs. Similarly, we want reduced linkage lengths as we want to reduce mass/weight of our linkage system. In similar fashion, we also want to ensure that our linkage system can sustain a force equivalent to that of the flashlight and anything used to fasten said light to the coupler.

Section 2: Design Selection

Criteria	Weight					
		Matt	Eli	Josh	Will	Matt Redesign
Total weight	4	0	0	0	+1	+1
Total angle traveled	3	0	-1	0	0	0
Transmission Angle Deviation	2	0	-1	-1	-1	0
Manufacturability	2	0	0	0	-1	+1
Total		0	-3	-2	0	+6

Table 2.1: Pugh Chart used to select the most efficient, manufacturing-friendly design. Matthew's first design was chosen as a template to work with. Improvements to Matthew's design reflect the group's decision to move forward past all other designs.

After looking at the data from Adams, we settled on these four criteria due to certain metrics given in the project specifications. We assigned a weight of 4 to the weight of the mechanism since a large part of our grade depends on how heavy it is. With that in mind, it became very evident that most of our designs were rather similar in weight. The one exception to this was Will's linkage design which weighed rather smaller than our clustered average, hence his score of +1. All in all, it became difficult to choose a design based solely on our weight since all five designs were almost identical in

magnitude. However, in principle, it still mattered to the group that we create a design that could be lower in net weight in order to achieve a higher score at the end of the semester. An excellent idea, suggested by Matthew, was to use a 3D printed coupler as the printed PLA would have a significantly lower density relative to aluminum.

We wanted the total angle traveled from position 1 to position 4 to be as small as possible to minimize travel time and decrease the amount of power needed to operate the mechanism faster. To optimize the mechanical advantage of the mechanism and thus its efficiency, we selected the deviation angle as criteria, but only gave it a weight of 2 since most of our deviation angles were similar throughout the positions. Lastly, we included manufacturability as criteria because choosing an easy to manufacture design would save a lot of time and minimize the possibility of manufacturing errors. It too was given a weight of 2 because all of our original linkages ended up requiring very similar manufacturing processes.

Our weighting criteria gave us a tie between Matt's and Will's couplers, so we decided to choose one to redesign. We settled on Matt's original design because it had better deviation angles and manufacturability compared to Will's, and we knew that we could make up for it having a higher weight than Will's by redesigning the coupler. After analyzing the redesign according to the Pugh chart criteria, the redesign of Matt's coupler won by a considerable margin. The redesigned coupler will be manufactured on the Polyjet Stratasys J850 in the Duderstat Center and will be considerably lighter compared to the aluminum couplers used in the other designs. The use of additive manufacturing will decrease the amount of time spent in the machining process and contribute to the redesign being the optimal choice.

	Matt	Eli	Josh	Will	Evan	Matt's Redesign
Input length (in)	8.47	8.67	7.19	5.42	7.88	8.47
Follower length (in)	8.04	8.95	6.24	5.22	7.72	8.04
Coupler length (in)	0.71	1.81	2.00	0.491	0.43	0.71
Power Required for 1 second linkage half-period (W)	0.160	0.107	0.123	.3164	0.0743	0.057
Angle Deviation (Position 1)	33.6	6.09*	38.95	58.07	19.73	33.6

Angle Deviation (Position 4)	28.4	46.77*	28.63	36.58	48.99	28.4
---	------	--------	-------	-------	-------	-------------

Table 2.2: General linkage design specifications, including the final draft linkage system (Matt's Redesign). All transmission angles < 90°. Max deviation angles < 60°

*Suspected high level of error in measured value, determined to be outlier data.

Section 3: Final Linkage Design and Analysis

Link	Final Length (in)
Input	8.47
Follower	8.04
Coupler	0.71

Table 3.1: Selected design lengths for linkages

Component	Mass (lb)
Input	0.11
Follower	0.11
Coupler/Flashlight Holder	0.04
Flashlight	0.13
Joints	0.08
Net Mass	0.47 lbm
Net Weight	0.47 lb

Table 3.2: Measured mass values for assembly components

	metric	Position 1	Position 2	Position 3	Position 4	Average
Transmission Angle	μ , degrees	123.6	93.4	78.2	61.6	89.2
Deviation	$ 90-\mu $, degrees	33.6	3.4	11.8	28.4	19.3

Table 3.3: Transmission angle & deviation for Position 1-4, and calculated average

Our targeted angle deviation was, in principle, 0° . The reason we desire to have the angle deviation this low is that it maximizes the mechanical advantage of the linkage system and decreases the amount of force we must apply at our input link. Despite the fact that our linkage system does not maximize our mechanical advantage in this sense, the maximum deviation observed from our preferred linkage design still falls under the hard cap of 60° .

	Distance from Bottom Left Corner of Plate (in)	
Link	X	Y
Ground Pivot A	6.028	3.498
Ground Pivot B	6.602	3.978

Table 3.4: Distance from bottom left of field to center of ground pivots

The Final CAD model should include:

1. A simplified model of the playing field
2. The flashlight (simplified model)
3. Fasteners, bearings, hard stops, and other important features

Outside of the linkage system itself, we decided to incorporate the 1" tube stock as the primary supply for our hard stop design. The goal is to drill through our sliced stock with a clearance hole, and have the head of the provided $\frac{3}{8}$ -16 shoulder screw sit on the surface while the tapped end is fastened through another clearance hole on the baseplate with a nut. To help protect the linkage from any potential damage from the force of slamming against the hard stop, we settled on using rubber/electrical tape to coat the exterior of the tube stock so as to absorb some of the force.

We had originally planned on mounting the base plate with 4 dowels and 2 additional shoulder screws, with said screws being placed on opposite sides of the base plate so as to secure the plate with equal tension. However, we realized that the height of the screws might interfere with our linkage's path of motion. As such, we decided to mount these screws on the back end of our base plate. The joints were designed based on the information slides provided for Labs 1 & 2 and were also

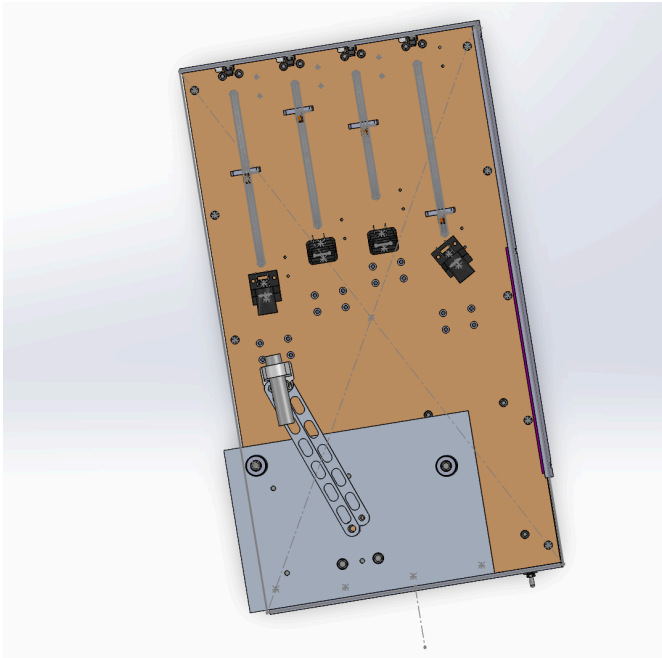


Figure 3.1.a: Linkage at Position 1

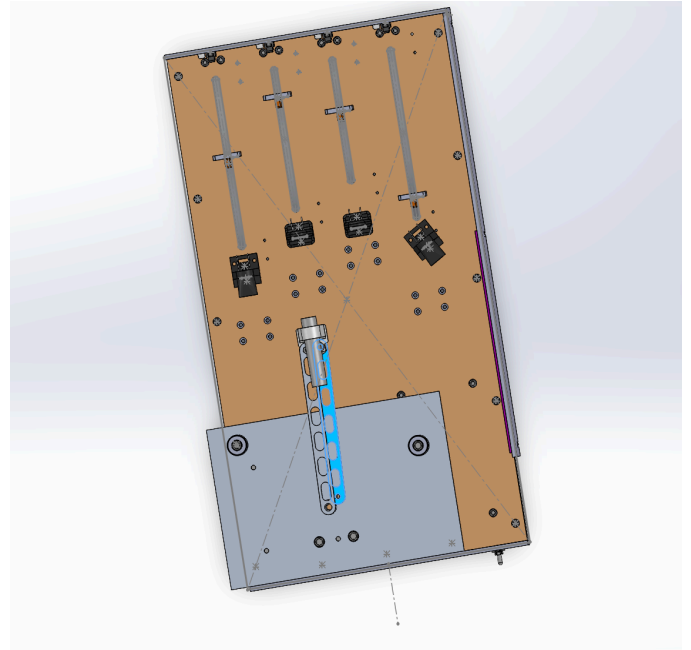


Figure 3.1.b: Linkage at Position 2

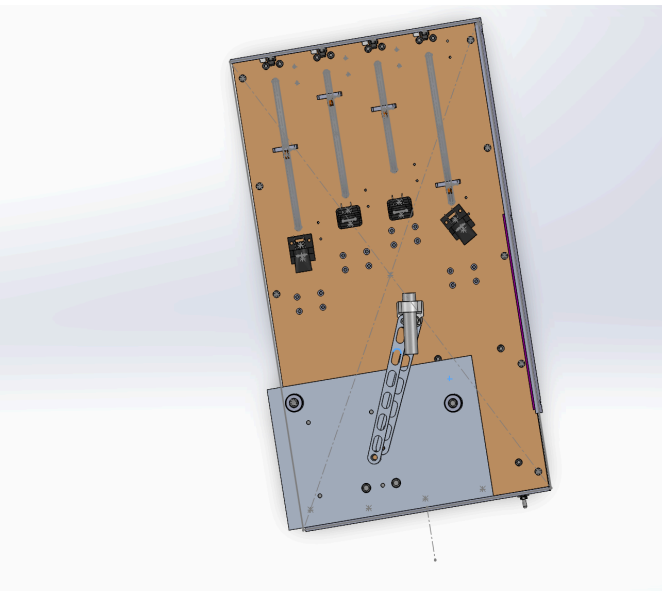


Figure 3.1.c: Linkage at Position 3

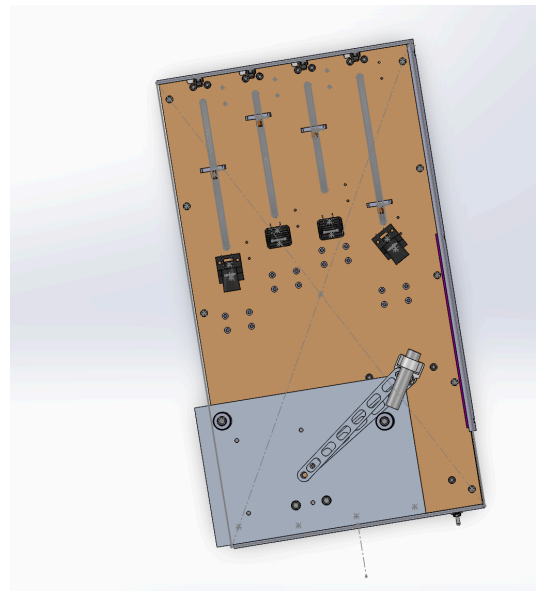


Figure 3.1.d: Linkage at Position 4

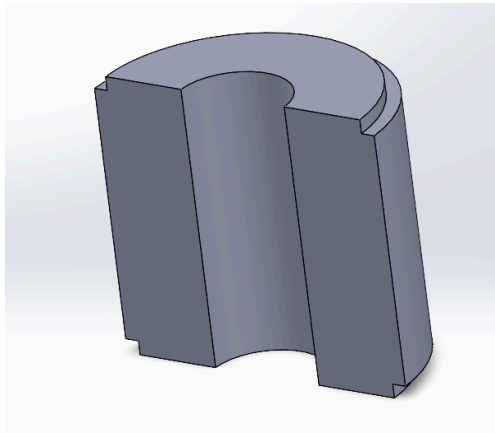


Figure 3.2: Cross Section of Hard Stop

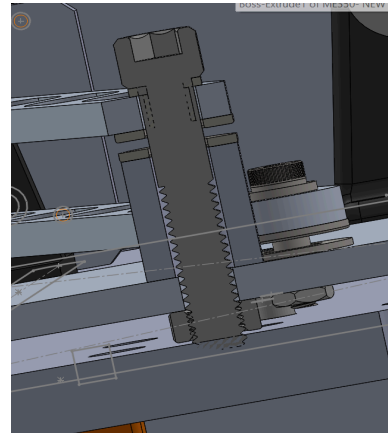


Figure 3.3: Cross Section of Input Joint

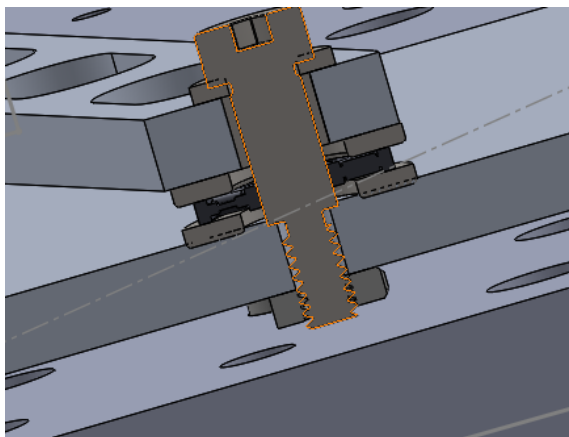


Figure 3.4: Cross Section of Follower Joint

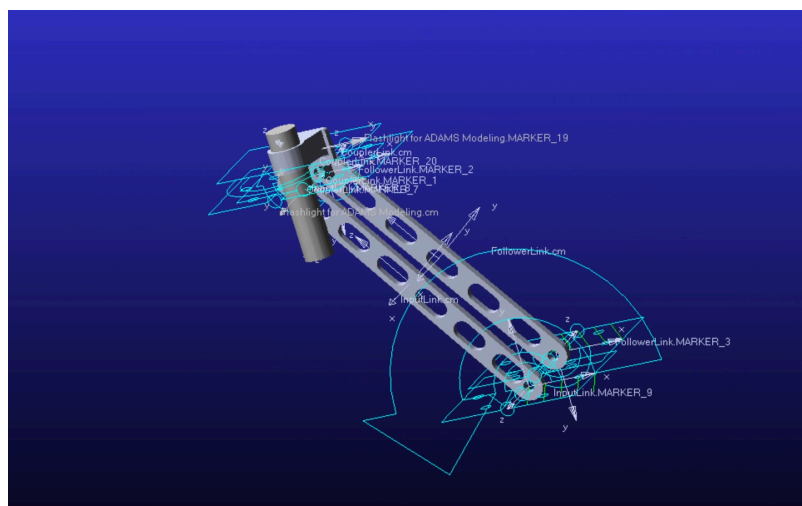


Figure 3.5 The linkage assembly in Adams

While we gained rather insightful data from running simulations as a result of uploading and configuring our linkage system in ADAMS, it is important to note that we must take our tracked values with a grain of salt. In the grand scheme of things, we can approximate our measurements to a rather high degree of accuracy. However, it must be noted that we neglect to take into account

any loss of motion or acceleration due to friction - specifically at the joints. Furthermore, we intentionally left out minor design details including, but not limited to: bushings, fasteners etc. While we made an assumption that leaving such components of the general assembly out of our kinematic analysis would result in minimal changes to our final data, there may be discrete differences in performance between our “perfect world” analysis and real life factors.

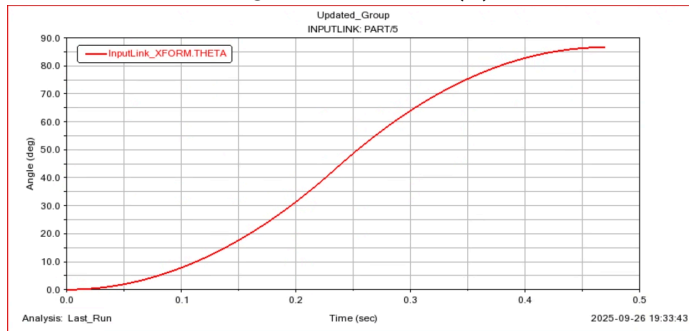
Angular Position (θ)

Figure 3.6.a: Min value of 0, max value of 90. Velocity is positive but decreasing. At the same time that the linkage switches direction, the slope flattens. As the magnitude of velocity increases, the slope increases and flattens out.

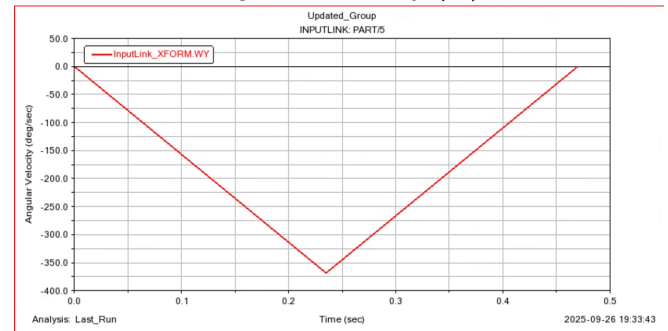
Angular Velocity (ω)

Figure 3.6.b: Max absolute value of 375deg/sec, min absolute value of 0.

Linkage maintains a linear and symmetrical speed throughout the entirety of its range of motion.

Power

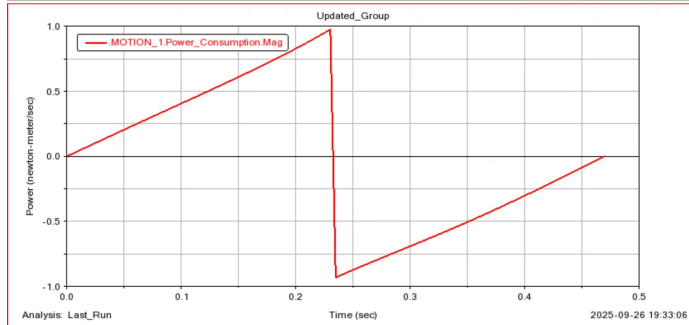


Figure 3.6.c: The max power consumption determined through ADAMS is .92 N*m/s and the min is -0.9 N*m/s.

The sudden change to negative power reflects the change in direction of linkage. Confirms design meets requirements of requiring less than 1 Watt.

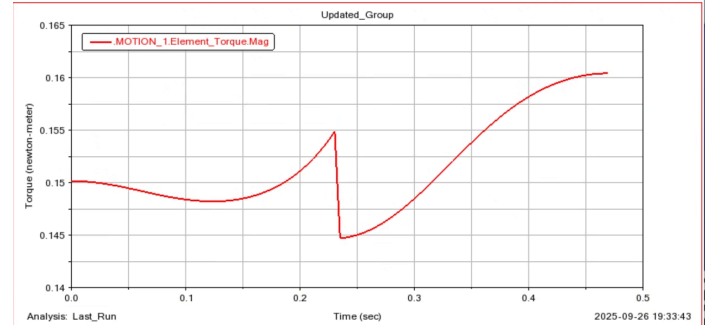
Input Torque (τ)

Figure 3.6.d: Minimum value of .125 Nm, maximum of 0.16 Nm

As the input link decelerates, the torque kicks in to counteract this motion. The midpoint $t = 0.25$ sec, the torque drops as the linkage switches direction. As the linkage accelerates, the torque picks up again.

Overall, the power consumption fell well beneath the 1W threshold when accounting for unit conversion. Most of our graphs looked rather predictable, at least in regard to the expected values we were hoping to achieve. One graph of note is that of the torques. Instead of being perfectly symmetrical or mirrored around the midpoint, the slopes on either side seem rather less consistent.

After contemplating potential reasons for this result, the conclusion was that the motion of our input is actually a piecewise function. As such, a discontinuity results in the torque plot.

Additionally, we tried sampling with different time values per the advice of the GSI at the review meeting. For the sake of simplicity with regard to our analysis, we chose a rather small time interval. However, after sampling larger time intervals, much of our data looked rather similar and reasonable.

Requirement	Met	Value
Acceptable range for transmission angle (μ): $30^\circ \leq \mu \leq 150^\circ$	Yes	See Table 2.2
Maximum angle deviation ($ 90-\mu $) must be less than 60°	Yes	33.6°
Input & follower linkages must each be less than 12" in length	Yes	8.47" & 8.04"
Maximum power required must be $< 1W$	Yes	0.92 W

Table 3.5: Checklist of requirements

Section 4: Motion Generator Revision

There were no major updates to the motion generation of our linkage. Our coupler design was changed, but due to the coupler being made of plastic, as well as the new design weighing practically the same, it would not have changed the physics of our mechanism in a significant way.

Section 5: Energy Conversion Introduction

There are multiple reasons to use transmissions in engineering applications. Transmissions allow for customization of the ratio of torque and speed, as well as the ability to transfer force across a distance. In the case of our transmission, we used a gear ratio of 1.6 to increase the torque output from the motor to the linkage. One of the issues with transmissions is the inverse relationship between torque and speed; increasing one will decrease the other by a proportional amount. In this

project, the limiting factors for the transmission are a limited budget, design feasibility/manufacturing time and restricted access to high-end 3D printers with quality materials (i.e not PLA). We could have picked a rather abstract design concept for our transmission, but using high-quality steel gears seemed to be the easiest option that minimized design and manufacturing times. One project goal is also to reduce weight. Using a gear system helps us attain a highly-functional linkage system while operating with a reduced mass, as opposed to something along the lines of a sophisticated pulley system.

Section 6: Transmission Ratio and Type Determination

Due to the relationship between torque and inertia, it is possible to estimate the desired transmission ratio as being proportional to the ratio of the moment of inertia of the linkage mechanism (output), and the moment of inertia of the motor (input). This is referred to as inertia matching, and is done to minimize torque with respect to the transmission ratio. The transmission ratio is found via Equation 1 where N = Transmission Ratio, I_L = Moment of inertia of load, and I_m = Moment of inertia of motor.

$$N = \sqrt{\frac{I_L}{I_m}}$$

Eq. 6.1: Transmission ratio specified with moments of inertia
[1] Equations in section 7 sourced from ME 350 Lab 5

The moment of inertia at the center of mass of each of the couplers is determined via the Solidworks model, and then, using the parallel axis theorem, Equations 2-5, the moments of inertia of the linkage components are determined.

$$\begin{aligned} I_{input} &= I_{input\ CG} + m_{input} \cdot \left(\frac{R_1}{2}\right)^2 \\ I_{output} &= I_{output\ CG} + m_{output} \cdot \left(\frac{R_4}{2}\right)^2 \\ I_{coupler} &= I_{coupler\ CG} + m_{coupler} \cdot (R_5)^2 \end{aligned}$$

Eq. 6.2, 6.3, 6.4: Moments of inertia for the linkage components

These equations are combined in Equation 6 to calculate the moment of inertia of the mechanism with regard to the ground pivot of the input. The moment of inertia for the given motor I_m is estimated as $25,000\text{ g} \cdot \text{cm}^2$

Inputting these values into Equation 1 yields the results summarized below.

$$\frac{1}{2}I_{total}\omega_{input}^2 = \frac{1}{2}I_{input}\omega_{input}^2 + \frac{1}{2}I_{coupler}\left(\frac{R_1\omega_{input}}{R_2}\right)^2 + \frac{1}{2}I_{output}\left(\frac{R_1R_3\omega_{input}}{R_2R_4}\right)^2$$

Eq 6.5: Total moment of inertia

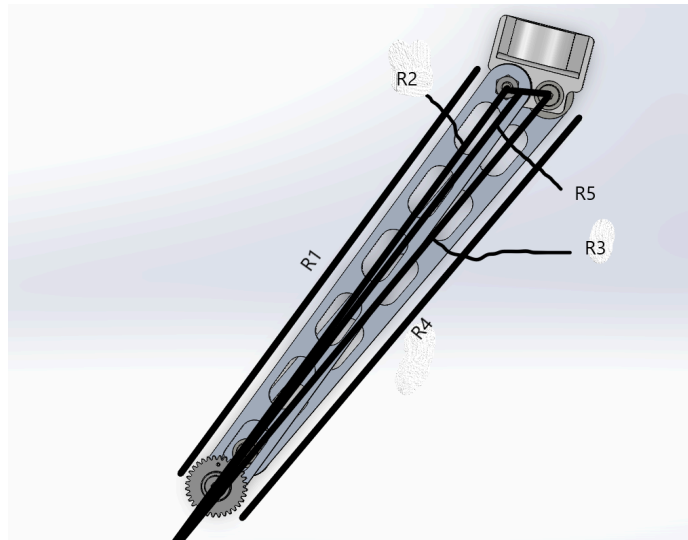


Figure 6.1 R values for the linkage (R_2 , R_3 , R_5 are very large in this position and meet at instant center (1,4))

***Final Transmission Ratio: 1.6**

Position	Transmission Angles	R1(cm)	R2	R3	R4	R5	Total Inertia	Transmission Ratio
1	123.6	21.52	1000	1000	20.63	1000	58563.96326	1.565624788
2	93.4	21.52	87.42	87.65	20.64	87.53	58728.61338	1.565624788
3	78.2	21.52	45.74	46.31	20.63	46.02	59317.32127	1.565624788
4	61.6	21.52	18.75	19.55	20.63	19.13	61279.5244	1.565624788

Table 6.1: Total Inertia and Transmission Ratio relative to Transmission Angle allowing us to approximate our ratio to **1.6**

Another method of selecting a transmission ratio is given from using light beam resolution. By measuring the range of angles where for each position half of the sensor is covered by the light beam

and selecting a desired encoder count range, in this case 30 counts, it is possible to use Equation 7 to solve for transmission ratio. Smallest difference was measured to be 3.52, and desired encoder count range was thus solved for a value of 1.598, very similar to the value from inertia matching.

$$\text{Desired Count Range} = \left(\frac{64 \text{ counts}}{1 \text{ rev}} \right) \cdot 30 \cdot \left(\frac{\text{smallest difference [deg]}}{\frac{360 \text{ degrees}}{1 \text{ rev}}} \right) \cdot N$$

Eq. 6.6

Gears, belts, and chains are three common transmission types used in industry. Gear transmissions give very accurate and efficient motion transfer, which makes them great for compact and high-torque applications. Belt drives use belts and pulleys and are quieter and cheaper but more likely to slip. Chain drives use links and sprockets, providing strong and reliable torque transfer but needing more maintenance. Our group chose to use gears because they are simple for design purposes, require minimal manufacturing and help us satisfy our goal to minimize linkage weight.

Section 7: Final Transmission Design

The given motor mount is attached to an L bracket made from ¼" Aluminium Angle Stock, which is in turn mounted to the baseplate on top of two spacers. The torque-transferring input joint consists of two spacers, washers, and the output gear, all on a shoulder screw. Our system utilized two stainless steel gears: both with a pressure angle of 20° and a gear pitch of 32. These two differed in that one was 32-tooth while the other was 20-tooth, which provided a gear ratio of 1.6. This allowed for ample torque transmission for our linkage. The gear attached to our input linkage is secured by an offset ⅛" spring pin that was hammered into concentrically aligned holes between our 32-tooth gear and our input link so as to reduce (ideally prevent) any relative rotation between the two. We avoided using any plastic pieces in our transmission system to avoid any warping or damage to the system.

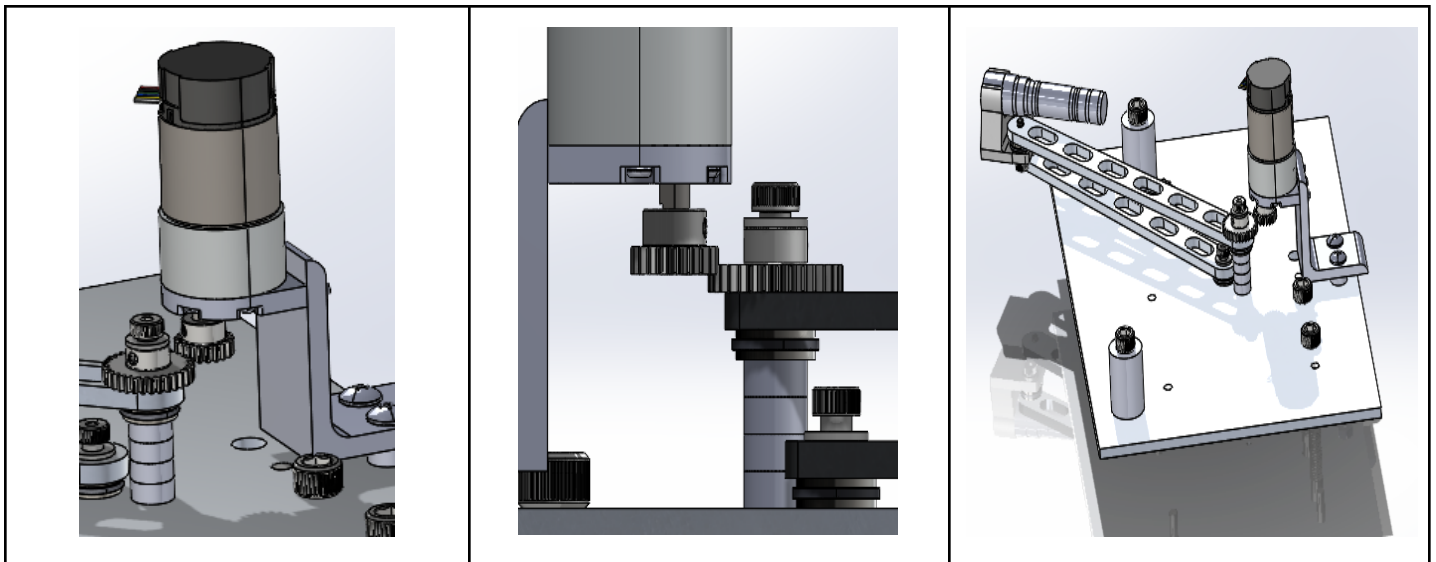


Figure 7.1 Close-up of motor and gear transmission	Figure 7.2 Cross-section of torque-transferring joint	Figure 7.3 Overall view of components
---	--	--

The total weight of the mechanism is estimated to be around 0.47lbs, with the gear estimated to weigh approximately 0.05lbs, totaling to around 0.52lbs. The L bracket, mounted with two philips head screws, is responsible for holding up the weight of both the motor and the input gear, meaning it is not accounted for in the calculations of the mechanism's weight. An assortment of spacers has also been used to ensure a proper height for our linkage. To add to the note of the face mismatch between the two gears, our team addressed a potential for our linkage to experience edge loading or shaft bending. However, because our gears are made of high-quality steel, we operated under the assumption that these potentially adverse effects might be rather marginal in the grand scheme of our linkage's functionality.

Section 8: Torque Transfer Analysis

*Assumptions: No loss of torque between the spring pin and input link and between motor and pinion.

In our design, torque is transferred in two distinct locations. Firstly, it is transferred from the motor to the transmission at the joint between the pinion and the motor shaft. This joint is secured by a set screw in the pinion that secures it to the shaft and allows for the motor torque to be transferred into the transmission. The other location is the joint between the gear and the input link. This joint is secured by a spring pin in the gear and input link and allows for the torque to be translated into angular motion in the linkage. Torque transfer analysis is performed to identify and eliminate any possible place in the transmission and linkage that could be subject to any sort of failure. Torque transfer analysis in this system begins with the first joint mentioned, the motor shaft and pinion. In this joint, the set screw is expected to transfer the torque the motor is generating to the pinion seen in this cross section of the joint.

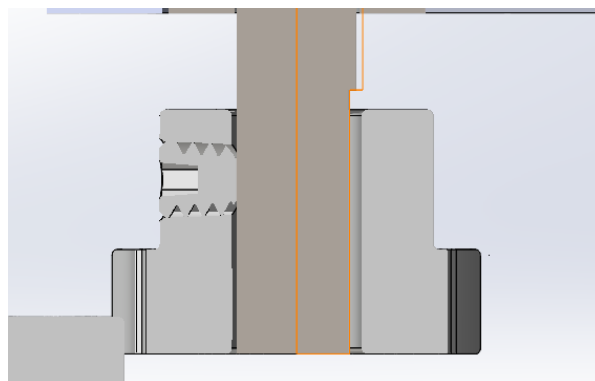


Figure 8.1: Cross section of the motor and pinion joint

$$T_{shaft} = 190 \text{ oz} \cdot \text{in} / 11.875 \text{ lb} \cdot \text{in}$$

$$D_{shaft} = 6 \text{ mm} / .23622 \text{ in}$$

$$R_{shaft} = .11811 \text{ in}$$

These values indicate that for this joint to effectively transfer torque to the transmission that set screw must be able to resist a torque of ~ 12 while the motor is producing its maximum torque. When comparing these values to the values provided in the tables of the DR2 Addendum we can see that the 6-36 set screw is able to exert a torsional holding power of 23 lb*in on the motor shaft. That value is far greater than the maximum torque produced by the motor at stall conditions showing that the set screw can indeed handle the load.

nom. size	seating torque inch-lbs.	axial holding power (pounds)	shaft diameter (shaft hardness Rc 15 to Rc 35)												
			1/16	3/32	1/8	5/32	3/16	7/32	1/4	5/16	3/8	7/16	1/2	9/16	
			torsional holding power inch-lbs.												
#0	1.0	50	1.5	2.3	3.1	3.9	4.7	5.4	6.2						
#1	1.8	65	2.0	3.0	4.0	5.0	6.1	7.1	8.1	10.0					
#2	1.8	85	2.6	4.0	5.3	6.6	8.0	9.3	10.6	13.2	16.0				
#3	5	120	3.2	5.6	7.5	9.3	11.3	13.0	15.0	18.7	22.5	26.3			
#4	5	160		7.5	10.0	12.5	15.0	17.5	20.0	25.0	30.0	35.0	40.0		
#5	10	200			12.5	15.6	18.7	21.8	25.0	31.2	37.5	43.7	50.0	56.2	
#6	10	250				19	23	27	31	39	47	55	62	70	
#8	20	385				30	36	42	48	60	72	84	96	108	
#10	36	540					51	59	68	84	101	118	135	152	
1/4	87	1,000							125	156	187	218	250	281	
5/16	165	1,500								234	280	327	375	421	
3/8	290	2,000									375	437	500	562	
7/16	430	2,500										545	625	702	
1/2	620	3,000											750	843	
9/16	620	3,500												985	
5/8	1,325	4,000													
3/4	2,400	5,000													
7/8	3,600	5,600													
1	5,000	6,500													

Figure 8.2: Set screw torsional and axial holding power

The next location that matters is the joint between the pinion and gear. Here, the stresses the gears are under can be directly calculated using the Lewis equation for bending stress and the contact stress equation that was introduced in lecture. For bending, we can see the stress each gear is under using the Lewis Equation for bending stress.

$$\sigma = \frac{K_v W^t P}{F Y}$$

Eq 8.1

Using values that can be found by doing a simple force analysis on the gears, we can determine that:

$$\sigma = \frac{(4.532\text{ lbf})(1.008525)(32\frac{\text{teeth}}{\text{in}})}{(.322)(.1875\text{ in})} = 2422.531\text{ psi}$$

$$\sigma = \frac{(4.532\text{ lbf})(1.0053)(32\frac{\text{teeth}}{\text{in}})}{(.364)(.1875\text{ in})} = 2136.156\text{ psi}$$

For the pinion and gear, respectively. Both of these are well under the AGMA allowable bending stress for 303 stainless steel of 17.5 ksi which shows that neither gear is at risk of failure in this mode. For contact stress we can use the equations introduced in lecture to determine the stress the gears are under.

$$r_1 = \frac{d_p \sin \phi}{2} \quad r_2 = \frac{d_G \sin \phi}{2} \quad C_p = \left[\frac{1}{\pi \left(\frac{1 - \nu_p^2}{E_p} + \frac{1 - \nu_G^2}{E_G} \right)} \right]^{1/2} \quad \sigma_c = -C_p \left[\frac{K_v W^t}{F \cos \phi} \left(\frac{1}{r_1} + \frac{1}{r_2} \right) \right]^{1/2}$$

Eq 8.2 Eq 8.3 Eq 8.4 Eq 8.5

$$r_1 = .625\text{ in} \cdot \frac{\sin(20)}{2} = 0.1068\text{ in} \quad r_2 = 1\text{ in} \cdot \frac{\sin(20)}{2} = 0.171\text{ in}$$

$$C_p = \sqrt{\frac{1}{\pi \left(\frac{1 - .25^2}{28\text{ Msi}} + \frac{1 - .25^2}{28\text{ Msi}} \right)}} = 2810$$

$$\sigma = -2180 \cdot \left[\left(\frac{1.0085 \cdot 4.532\text{ lbf}}{0.1875 \cdot \cos(20)} \right) \cdot \left(\frac{1}{.1068} + \frac{1}{.171} \right) \right]^{\frac{1}{2}} = -55818.49\text{ psi}$$

Comparing this calculated value to the AGMA allowable value of 1,090Msi for 303 stainless steel we can see that the gears are not at risk of failure in this mode either. The final location where torque transfer analysis must be performed is the joint between the gear and the input link. In this joint a steel spring pin is expected to transfer the torque from the gear onto the input link of the linkage seen in this cross section below.

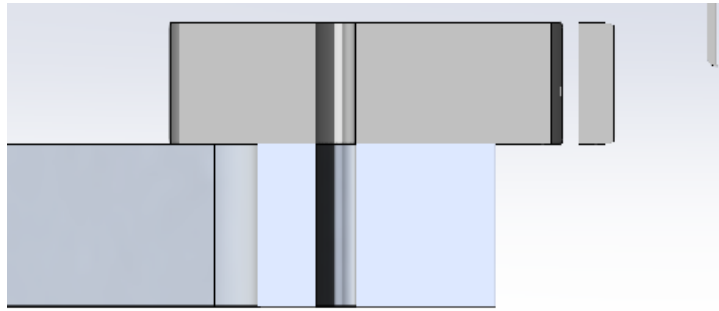


Figure 8.2: Cross section of input link and gear.

Here, again assuming that the motor is generating its maximum torque, we can see that the gear in this joint must transfer a torque of $\sim 11.9\text{lb}\cdot\text{in}$ to the input link. The spring pin here is subject to shear force from both the gear and the input link. These forces can be simply calculated.

$$r = \frac{0.125\text{in}}{2} = 0.0625\text{in}$$

$$F = \frac{T}{r} = \frac{11.9}{0.0625} = 190\text{lb}$$

$$\tau = \frac{F}{A} = \frac{2F}{\pi \cdot d^2} = \frac{2 \cdot 190\text{lb}}{\pi \cdot 0.0125^2} = 7740\text{psi}$$

Comparing this calculated value to the allowable shear stress of steel, $\sim 60\text{ksi}$, it is clear that the spring pin is under no risk of failure due to shear.

Section 9: Motor Controls

Motor controls are necessary for providing precise and repeatable movements for the linkage system. Ideally, the motor would be directed to move a certain number of encoder counts and move to the desired location every time; however, friction and inertia need to be accounted for. Motor controls are implemented so the motor not only moves to the desired position, but also corrects its motion in real time.

The sensors used in the control system of this project were a hall encoder, a proximity sensor, and a limit switch. The limit switch is used to calibrate the linkage. The quality of controls is of the utmost importance for this project, as they enable the linkage to move to its desired position.

The tuning of the PID controller is one of the most important parts of the system. The PID is a closed-loop controller for the motor system; it uses a motor hall encoder to track the motor shaft position and direction, which can be used to calculate the linkage velocity.

The PWM control is used to control the velocity of the motor, utilizing the duty cycle, which rapidly turns the supply voltage on and off to reach the desired speed. The higher the duty cycle, the higher the speed of the motor. Both of these systems are essential to make sure the linkage moves with both accuracy and speed—qualities that are essential to maximize points.

Proximity sensors use infrared light to determine how far away the zombies are from the targets. Some drawbacks with these sensors are that they don't work when the object being sensed is very close to the sensor. These work by emitting an IR signal and then detecting how long it takes to bounce back to the phototransistor. Where they struggle, though, is with picking up noise from ambient light from around the room, which can mess with the readings. Additionally, if the zombie is too close to the sensor, it can incorrectly read the distance.

Section 10: Encoder Counts, Proximity Sensor Thresholds, and Controller Gains

To determine the desired positions of the encoders, the supplied Encoder Test script was used. When this code is active, the current encoder position of the linkage is constantly printed to the console. The position can be zeroed when the follower link hits the limit/snap action switch. For each target, the linkage was moved, such that the beam of the flashlight was directly on the target. Because the linkage generated a loss of precision from the long shoulder bolt moving within its tapped hole, after finding the encoder position of each target from the counterclockwise (positive) direction, the encoder position of each target was found from the clockwise (negative) direction. These values were averaged so that no matter what direction the linkage lit the targets from, they would sense the light. This process was done every time our limit switch was adjusted.

The proximity sensor ranges were found using a script. Once the script was run, the values of all 4 proximity sensors were constantly checked for 44.5 seconds. The highest and lowest values of each proximity sensor were output. When the script is run with the board running round one simultaneously, the ranges of all 4 proximity sensors were found.

For our friction compensation voltage, a similar tuning script that applied a desired voltage to the linkage was used. We moved our linkage to the position with the highest transmission angle deviation because this is where the minimum voltage to move the linkage is the highest. In the script, we increased the applied voltage until the linkage moved, and recorded that as our friction voltage.

Our controller gains were initially tuned by increasing the proportional gain until the linkage oscillated, and then increasing the derivative gain until it didn't. Finally, these values were divided by 2 to get our Kd and Kp. The integral gain was set to 0.001 as per the given instructions. This was changed when it was realized that the motor supplied was a 50:1 motor. Because our linkage had a higher transmission ratio than necessary, the values were returned to their original values from before being divided by 2.

Variable Name in Arduino Sketch File	Purpose of Variable	Device	Actual Value Used During Testing
<i>TARGET_1_POSITION</i>	Position for Target #1	Motor	1305
<i>TARGET_2_POSITION</i>	Position for Target #2	Motor	938
<i>TARGET_3_POSITION</i>	Position for Target #3	Motor	645
<i>TARGET_4_POSITION</i>	Position for Target #4	Motor	40
K_p	Proportional Gain	Motor	0.250
K_i	Integral Gain	Motor	0.001
K_d	Derivative Gain	Motor	0.020
<i>FRICTION_COMP_VOLTAGE</i>	Friction Compensation Voltage	Motor	3V
<i>PROXIMITYSENSE1_MAX</i>	Sensor Maximum Threshold for Target #1	Proximity Sensor	586
<i>PROXIMITYSENSE1_MIN</i>	Sensor Minimum Threshold for Target #1	Proximity Sensor	101
<i>PROXIMITYSENSE2_MAX</i>	Sensor Maximum Threshold for Target #2	Proximity Sensor	573
<i>PROXIMITYSENSE2_MIN</i>	Sensor Minimum	Proximity Sensor	142

	Threshold for Target #2		
<i>PROXIMITYSENSE3_MAX</i>	Sensor Maximum Threshold for Target #3	Proximity Sensor	587
<i>PROXIMITYSENSE3_MIN</i>	Sensor Minimum Threshold for Target #3	Proximity Sensor	142
<i>PROXIMITYSENSE4_MAX</i>	Sensor Maximum Threshold for Target #4	Proximity Sensor	590
<i>PROXIMITYSENSE4_MIN</i>	Sensor Minimum Threshold for Target #4	Proximity Sensor	95

Table 10.1: Summary of Variables in Arduino Code

Section 11: Final Testing Results/Discussion:

Our setup included our linkage assembly, Arduino/H-bridge plate, and the playing field provided in the Mechatronic Lab. We made multiple adjustments to our limit switch orientation to ensure that the linkage was connecting every time as intended. Additionally, we had to recalibrate our target positions after a few runs based on design modifications.

Our initial score of 120 was significantly higher than many other groups'. We discovered that our coupler could only go ten runs before fracturing. As such, we reprinted the coupler with an alternative orientation that allowed for additional weight support for the flashlight clasp. Shortly afterward, our score dropped rather drastically compared to our initial run. We made adjustments to the Arduino code, orientation of the flashlight, and several other minor things before it was discovered that one of the pins used to connect the playing field to our circuit had been sheared off. We promptly replaced it with an alternative playing field.

We returned to scoring rather high, relative to our initial run, and attempted to troubleshoot issues we had with our linkage dropping targets, specifically in the third round. Seeing as flashlight orientation was part of our problem, we angled our flashlight slightly downwards and increased the Kp value within our Arduino code. However, we ran into an issue where targets would still drop as a result of the backend of the flashlight bumping into the hardstop, thus pushing it in and reducing the intensity of the light beam. We created a countersink hole in both hard stops and brought both bodies down in length so as to avoid any potential for reduced accuracy.

Our final score hit the mid 250's, which was higher than many other groups had achieved who were facing similar challenges. Shortly after testing, it became apparent that our linkage was still not fast enough to keep up with the targets, and no determination was made immediately as to what the cause could be. Eventually, it was revealed that we were given an incorrect motor to drive our linkage system, operating with a 50:1 internal gear ratio as opposed to the project-specified 30:1. This meant that our linkage operated at 60% of the designed and desired speed. This was the underlying roadblock that our linkage had been experiencing for most of our testing runs.

Our group feels that a score in the mid 250's is rather competitive with much of the class. With that said, it is slightly disappointing that the linkage Team 24 designed and manufactured was unable to highlight the benefits of its design to the maximum. All in all, our group was rather satisfied with our results, given the disadvantage that we were faced with.

Trial #	Score	Issue(s)	Fix(es)
1	120	→ Target 4 registration inconsistency → Poor limit switch activation	→ Re-printed coupler on alternative face → Adjusted limit switch orientation
2	87	→ Targets 1 & 2 registration inconsistency	→ Replaced flashlight battery, recalibrated Target locations, replaced playing field
3	180	→ Linkage moving slower than anticipated	→ Increased Kp value
4	253	→ Extended flashlight hit hardstops → Wrong motor driving linkage	→ Brought hardstops down in length by ¼" in length, added countersink hole

Table 11.1: Table of significant trial runs with issues encountered and resulting adjustments

Section 12: Design Critique & Evaluation

One element of the project that was rather enjoyable was that one of our most difficult designs, the coupler, was created using the 3D printers that were provided to us in the X50 Assembly Room; this helped us reduce linkage weight and saved time on other, more complicated alternative methods for manufacturing. Additionally, our base plate design contrasted with many other 350 groups in that ours was relatively large. We did this so as to maintain stability and reduce additional oscillation/vibration on our linkage. One thing we failed to account for was the flashlight being extended out and striking our hardstops; the flashlight would hit the stops, which would reduce the accuracy of the flashlight. In response, we milled countersink holes into the hardstops and brought them down in height.

Our strategy involved prioritizing targets 2 and 3 as they had the shortest distance required to travel before the zombie hit. We also prioritized target 4 over target 1 for the sole reason that our linkage was inconsistent in properly sending it back, despite light being perfectly reflected through the target sensor. Overall, the strategy worked rather well.

Our design did not act as ideally as we would have liked. However, we discovered that the motor driving our linkage system was operating with the 50:1 internal gear ratio as opposed to the design-specified 30:1 internal gear ratio. This helped us increase the accuracy of our flashlight during the first two rounds, but restricted our linkage's ability to shift efficiently during the third round. One minor adjustment we might make in the future is replacing the screw that fastens the input link with a dowel in order to reduce play between the motor and linkage itself. All in all, I think we would have had one of the highest scoring linkage systems in the class had we been given the correct motor.

Friction definitely played a part in restricting our linkage's ability to perform at its maximum capabilities - at least according to theory. However, we found that lubricating the meshing point of our gears helped us to reduce the adverse effects on our system and brought up our score relative to some of our preliminary runs.

Appendix A: Assembly Manual

Step 1: Press fit bushing (2) into the untapped end of the input link (1). Repeat for the follower link



Figure A.1

Step 2: Press fit bushings (2) into the holes on the coupler (1)

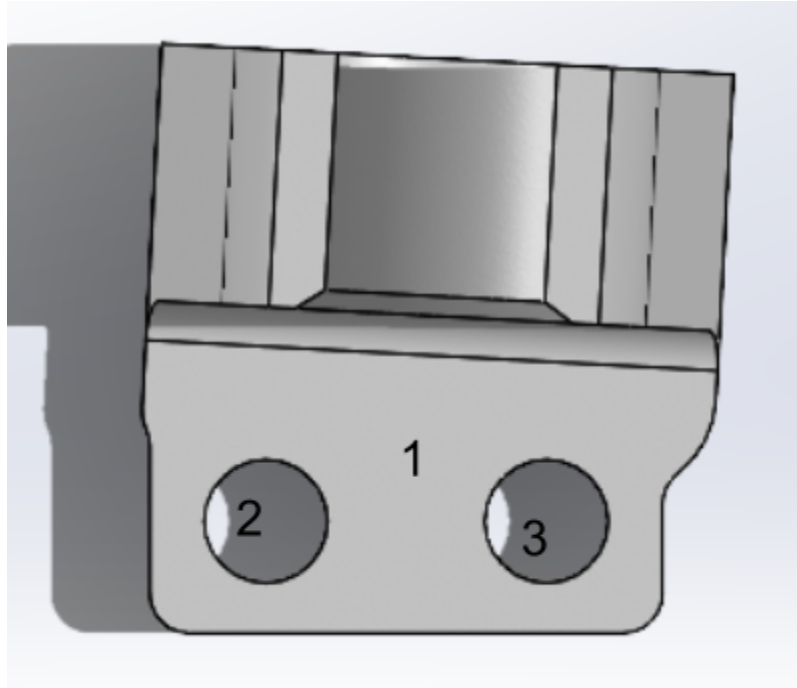


Figure A.2

Step 3: Slide washer (4), coupler (1), and thrust bearing(5) onto a shoulder screw(3). Tighten the shoulder screw into the tapped follower(2). Top with nut(6)

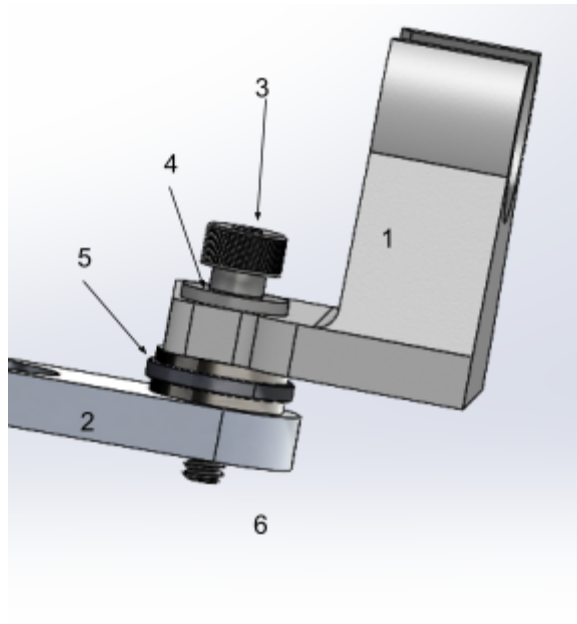


Figure A.3

Step 4: Repeat Step 3, but upside down on the coupler (1), and using the input link (2) instead of the follower link. (shown without nut)

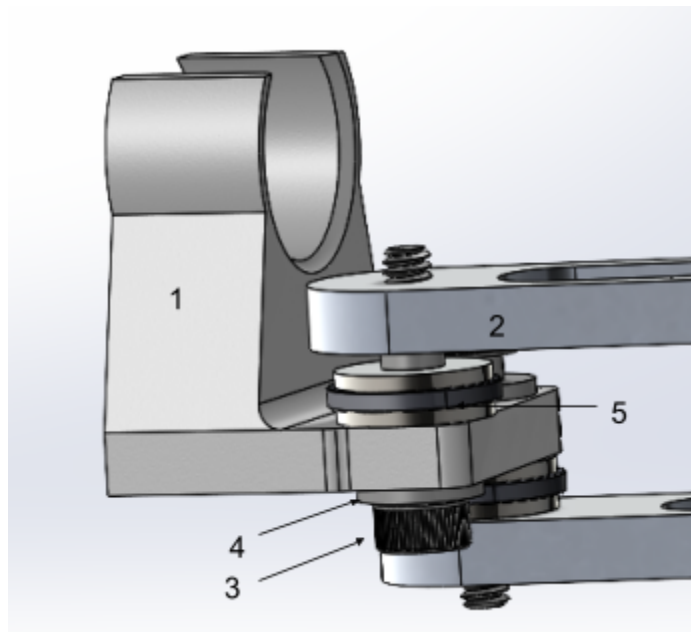


Figure A.4

Step 5: Press fit the 4 dowels (2) into the baseplate (1)

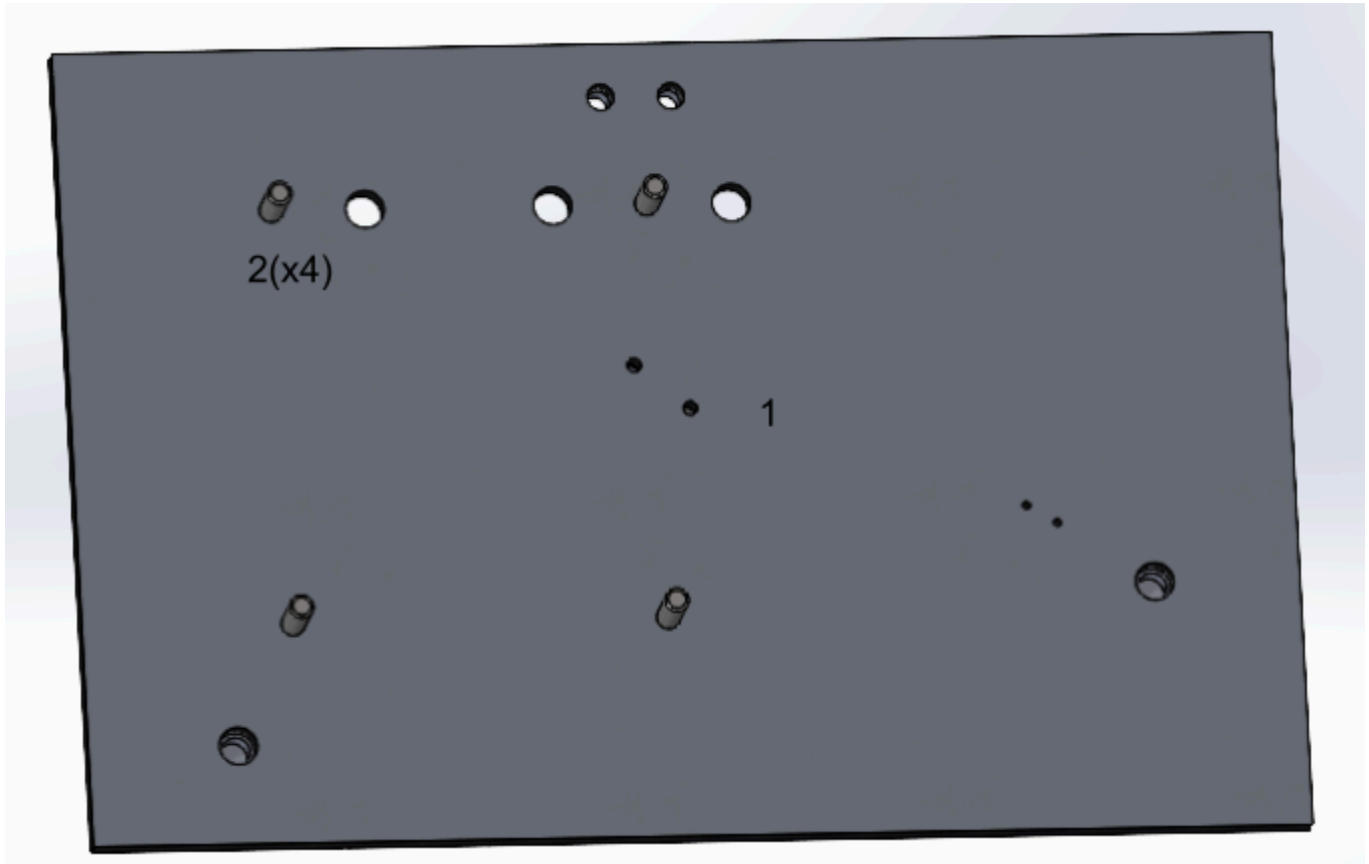


Figure A.5

Step 6: Repeat Step 3, but the coupler is now the follower (1) and the follower is now the baseplate

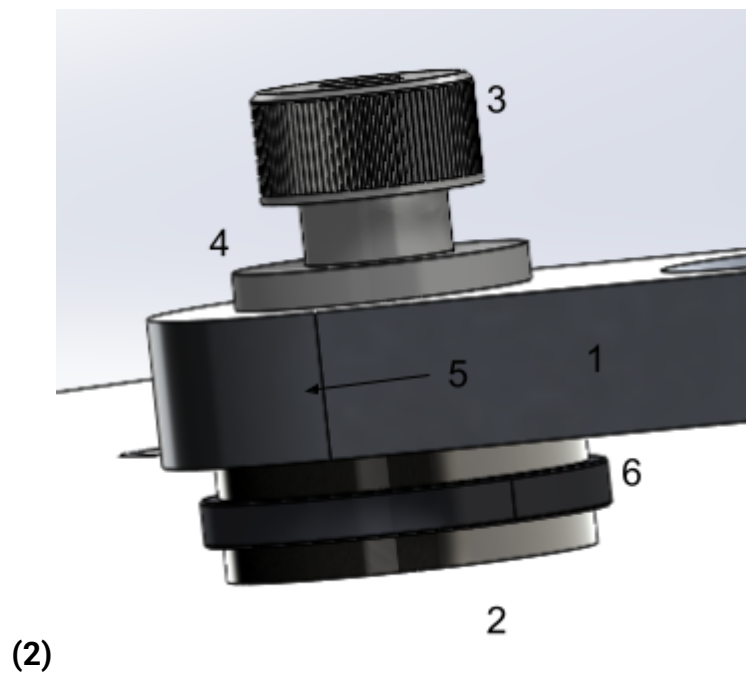


Figure A.6

Step 7: Repeat Step 6, but with the input link (3) instead of the follower. Before screwing the shoulder screw (1) into the baseplate (7), put the 3D printed spacer (5) onto the screw.

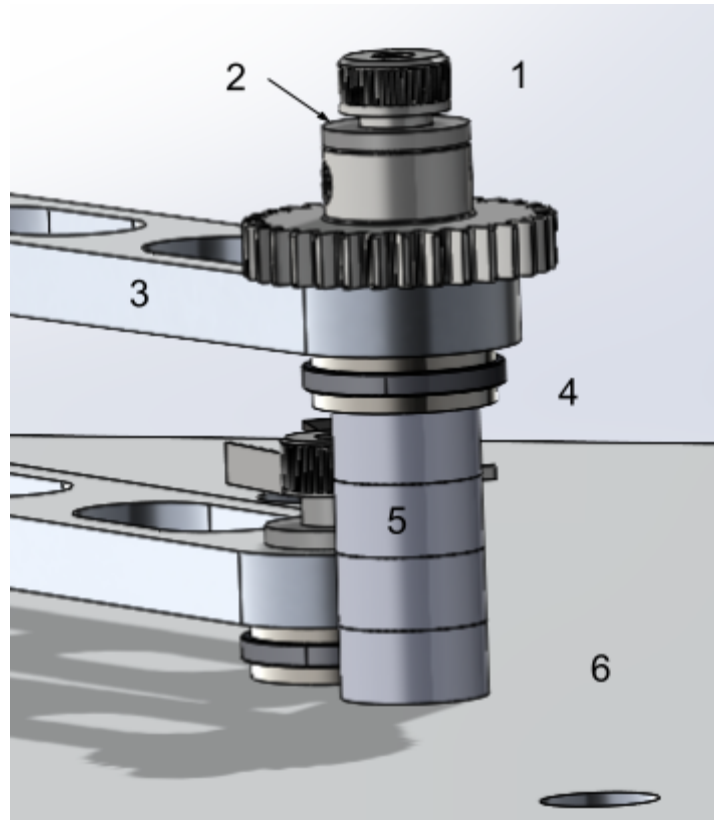


Figure A.7

Step 8: Attach the hard stops (2) to the baseplate with shoulder screws (1) and nuts (3)

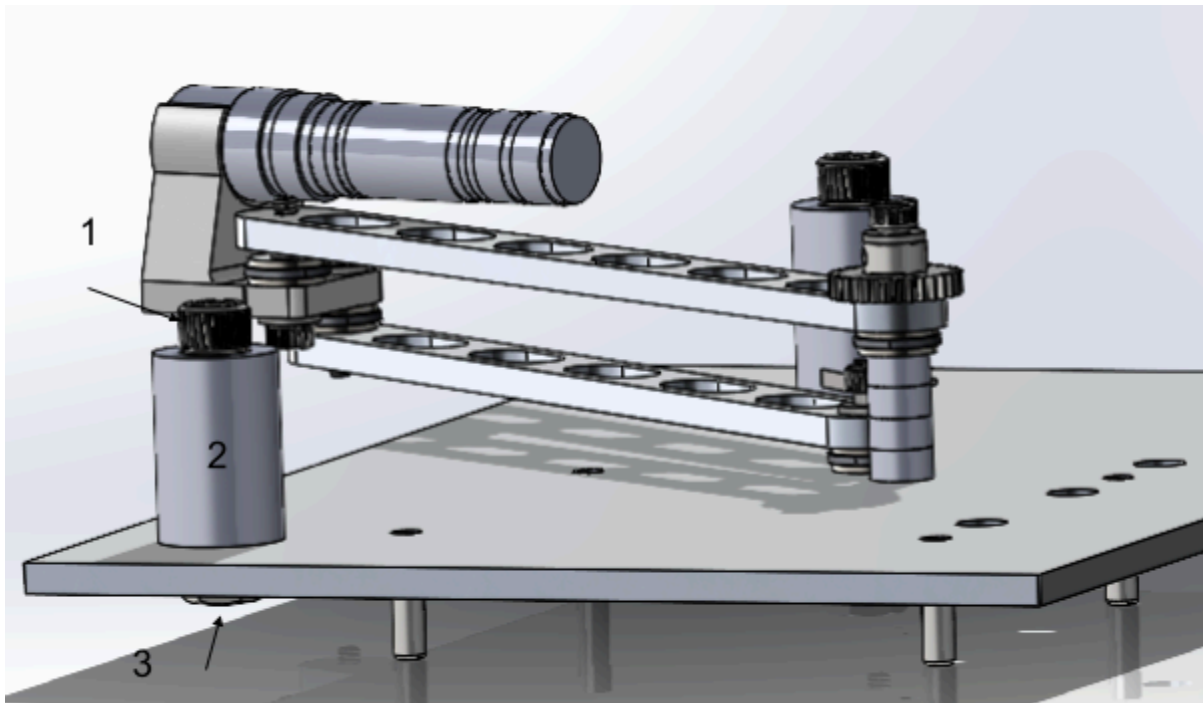


Figure A.8

Step 9: Screw shoulder screws (2) into baseplate (1) and game arena (not shown)

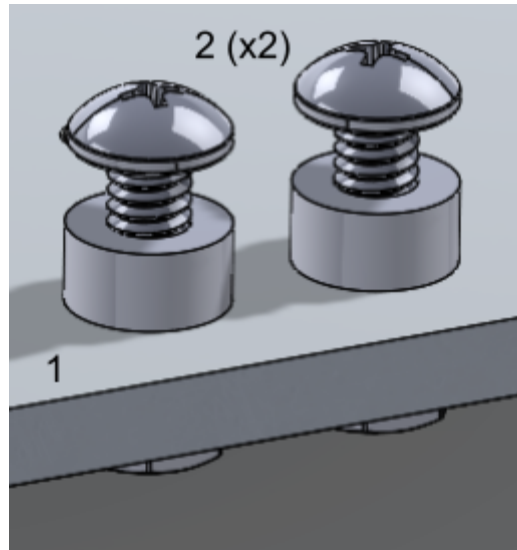


Figure A.9

Step 10: Push the flashlight (1) into the coupler (2)

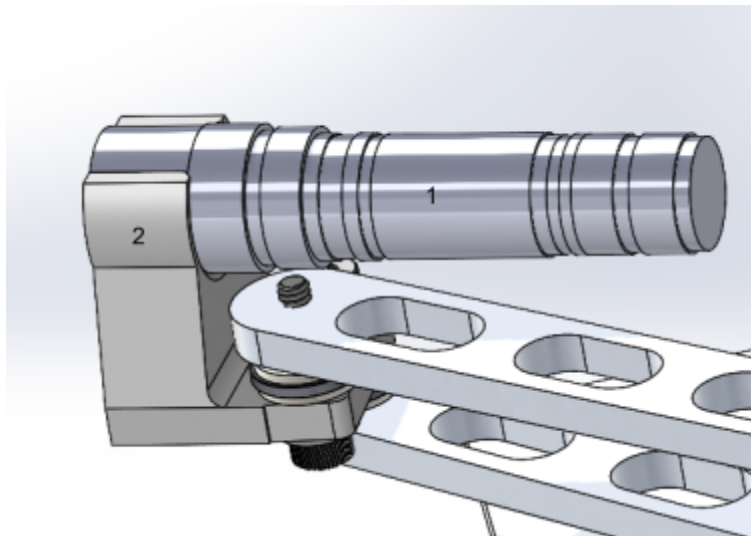


Figure A.10

Appendix B: Bill of Materials for the Final Design

								Design/CAD	Drawing/Plan	Machining
1	Input	Aluminum	9.27" x 0.8" x 0.25"	350 Kit	1	\$0		Matt Gordon	Evan Link	Will/Matt
2	Follower	Aluminum	8.8" x 0.0" x 0.25"	350 Kit	1	\$0		Matt Gordon	Evan Link	Will/Matt
3	Coupler	PLA	1.49" x 1.65" x 1.79"	3D Printers	1	\$5		Matt Gordon	Evan Link	Matt/Will
4	Baseplate	Aluminum	12.97" x 8.60" x .25"	350 Kit	1	\$0		Will Pepper	Evan Link	Will/Matt
5	Hardstop	Aluminum	Dia 1" x 1.50"	350 Kit	2	\$0		Will Pepper	Evan Link	Eli
6	Sleeve Bearing	Bronze	Dia 0.315" x .25"	350 Kit	4	\$0		McMaster	-	-
7	Oil-Embedded B	Bronze	1/4" Shaft Diameter, 5/8" OD, 1/	350 Kit	8	\$0		McMaster	-	-
8	Oil-Embedded B	Bronze	1/4" Shaft Diameter, 1/2" OD, 1/	McMaster	4	\$13		McMaster	-	-
9	Shoulder Screw	Steel Alloy	1/4" Shoulder Diameter, 5/8" Sh	X50 Room	3	\$0		McMaster	-	-
10	Short-Thread All	Steel Alloy	1/4" Shoulder Diameter, 2" Shou	X50 Room	1	\$0		McMaster	-	-
11	M3x8 Philips Scr	Stainless Steel	M3 x 0.5mm Thread, 8mm Long	X50 Room	5	\$0		McMaster	-	-
12	Black-Oxide Allo	Black Oxide Alloy Steel	3/8"-16 Thread Size, 2" Long, Pa	X50 Room	4	\$0		McMaster	-	-
13	Zinc Plated Steel	Zinc-Plated Steel	1/4"-20 Thread, 3/4" Long	X50 Room	2	\$0		McMaster	Matt	Eli
14	Needle Thrust B	Alloy Steel	0.25" ID, .687" OD x .078" long	350 Kit	4	\$0		Grainger	-	-
15	Metal Gear - 20	303 Stainless Steel	0.625" PD, .25" ID	McMaster	1	\$25		McMaster	-	-
16	Metal Gear - 20	303 Stainless Steel	1" PD, 0.25" ID	McMaster	1	\$27		McMaster	-	-
17	Toggle Switch	-	1.6" x 1.13"	350 Kit	1	\$0			-	-
18	Medium-Strengt	Steel	Grade 5, 3/8"-16 Thread Size	McMaster	2	\$0		McMaster	-	-
19	Steel Hex Nut - 5	Steel	SAE Grade 5, 1/4"-20 Thread Size	McMaster	2	\$0		McMaster	-	-
20	1/8" Spring Pin	Steel	1/16"	X50 Room	1	\$0		McMaster	-	-
20	L Bracket	Aluminum	2.5"x2"x1.45"	350 Kit	1	\$0		Matt Gordon	Will	Josh
21	Spacer (Limit)	PLA		3D Printer	1	\$0		Matt Gordon	Will	Will/Matt
22	Spacer	Nylon	1/4" ID, 1/2" OD x 0.25"	X50 Room	6	\$0		Matt Gordon	-	-

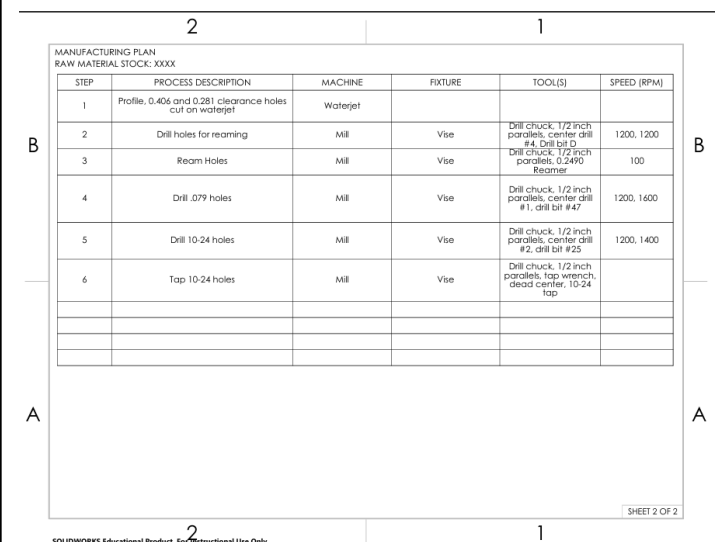
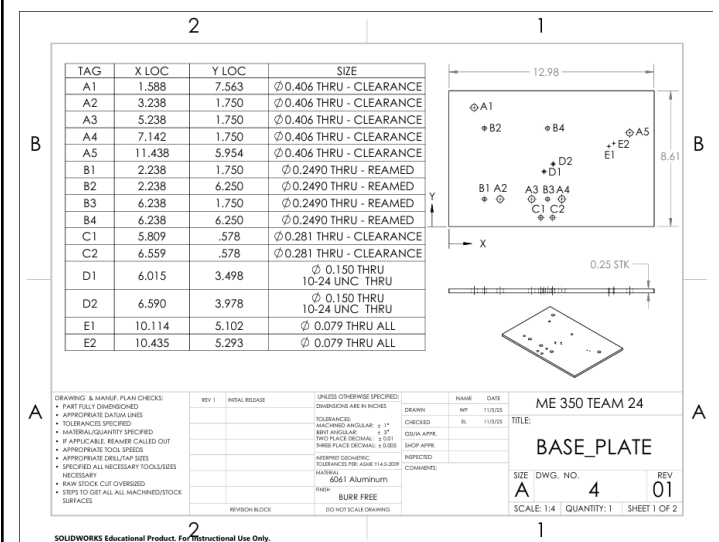
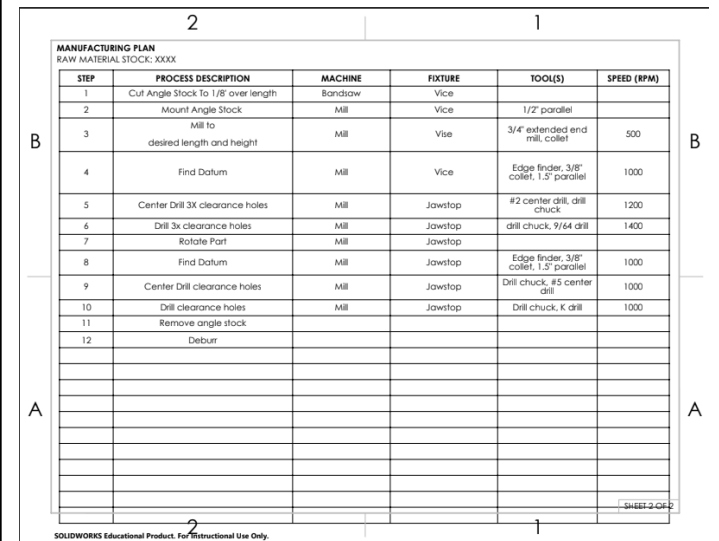
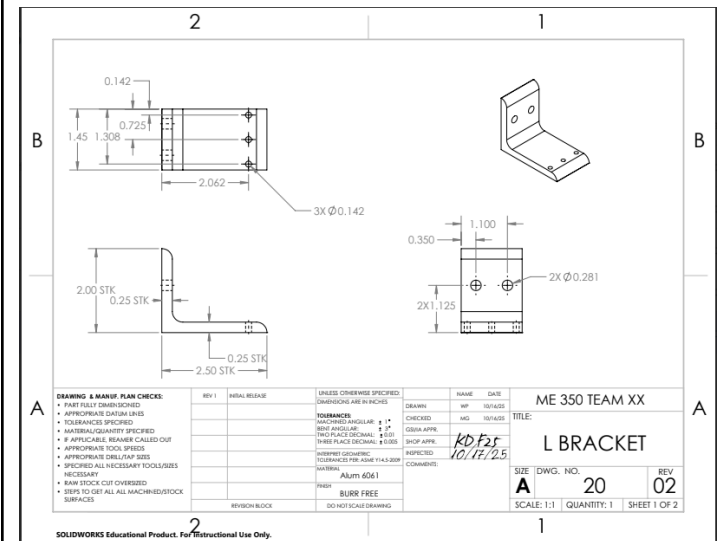
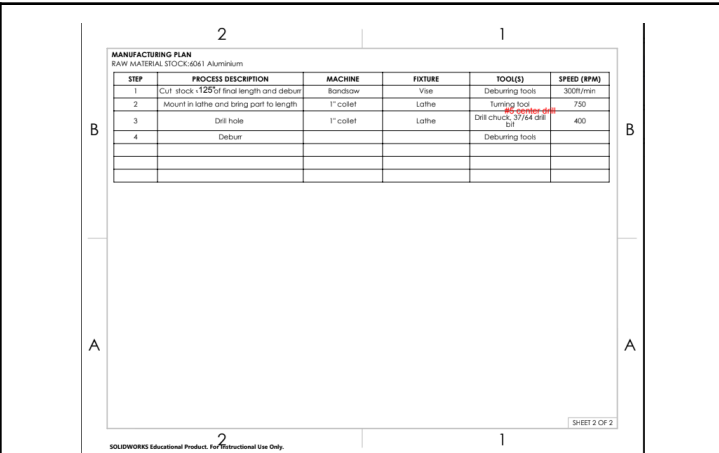
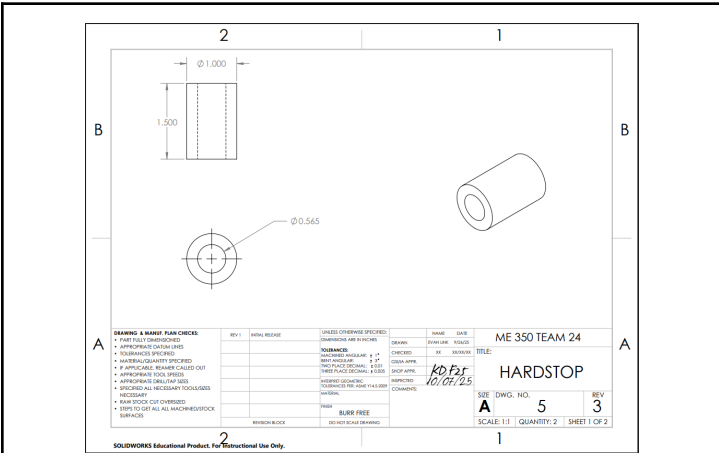
Figure B.1: BOM for Team 24

Purchased Part No.	Qty	Price/Unit	Supplier Part No.	Cost
15	1	\$25.02	<u>6832K52</u>	\$25.02
16	1	\$27.00	<u>6832K54</u>	\$27.00
9	8	\$1.60	<u>5906K562</u>	\$12.80
				\$0.00
				\$0.00
				\$0.00
				\$0.00
				\$0.00
				\$0.00
				\$0.00
				\$0.00
				\$0.00
Shipping	1	\$10.15		\$10.15
Tax	1	\$3.12		\$3.12
Total Cost	\$78.09			
*Don't forget about shipping/taxes				

Purchase Part Links
https://www.mcmaster.com/6832
https://www.mcmaster.com/6832
https://www.mcmaster.com/produ

Figure B.2: Purchased Part List + Order Links

Appendix C: Approval Packages



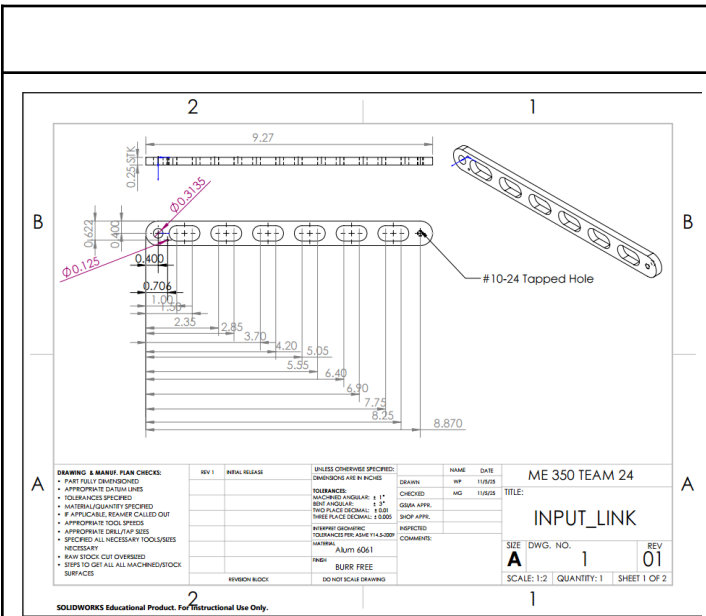


Figure C.4.A Drawing for Input

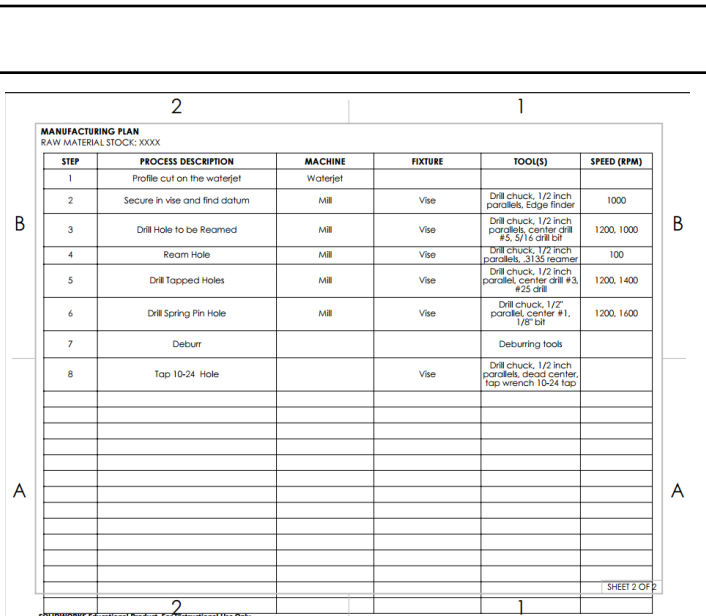


Figure C.4.B Manufacturing Plan for Input

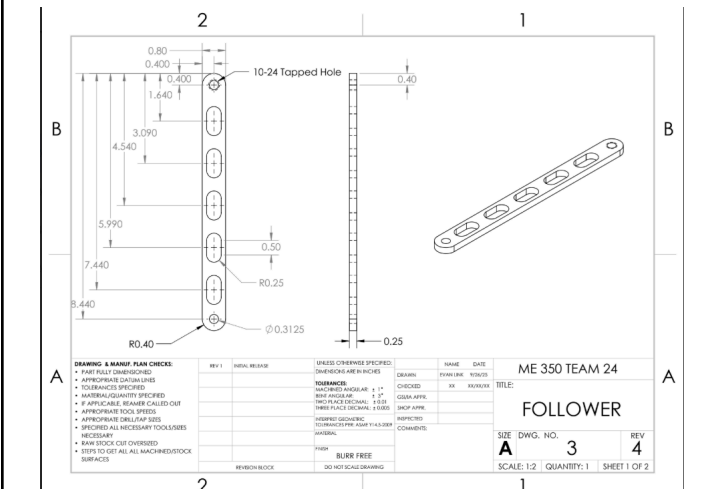


Figure C.5.A Drawing for Follower

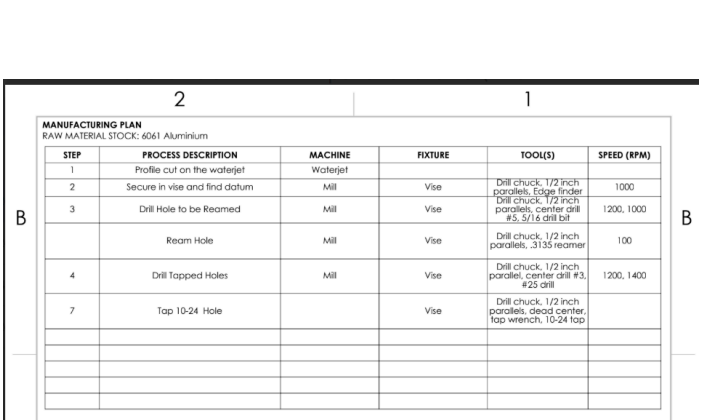


Figure C.5.B Manufacturing Plan for Follower

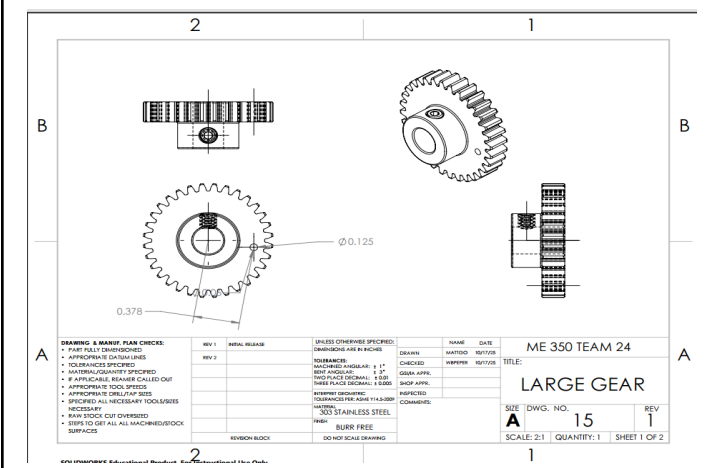


Figure C.6.A Drawing for Gear

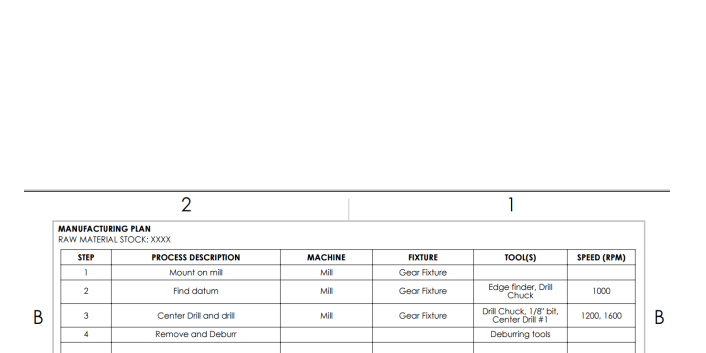


Figure C.6.B Manufacturing Plan for Gear

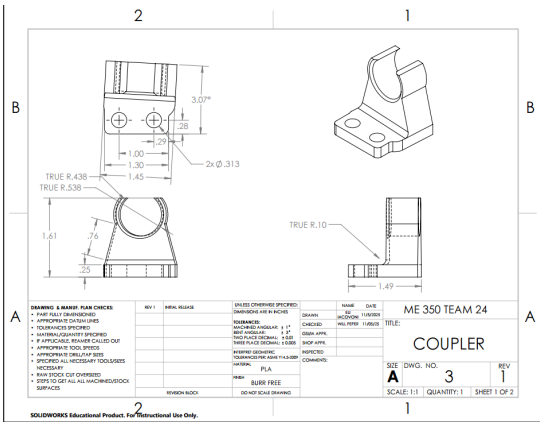


Figure C.7.A Drawing for Coupler

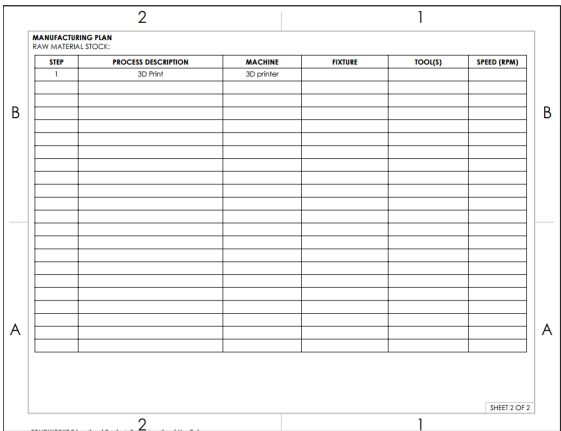


Figure C.7.B Manufacturing Plan for Coupler

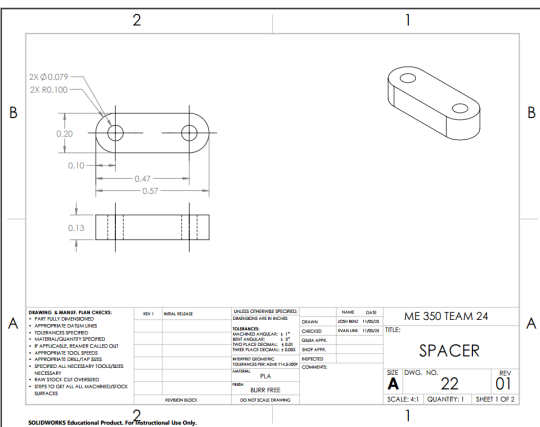


Figure C.8.A Drawing for Spacer(Limit)

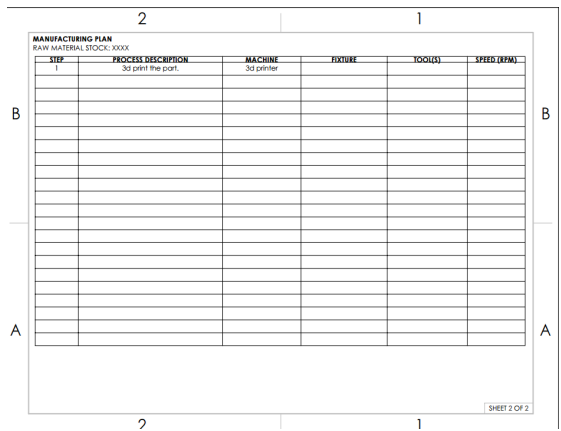


Figure C.8.B Manufacturing Plan for Spacer(Limit)

Appendix D: Wiring Diagram, Arduino Code Changes

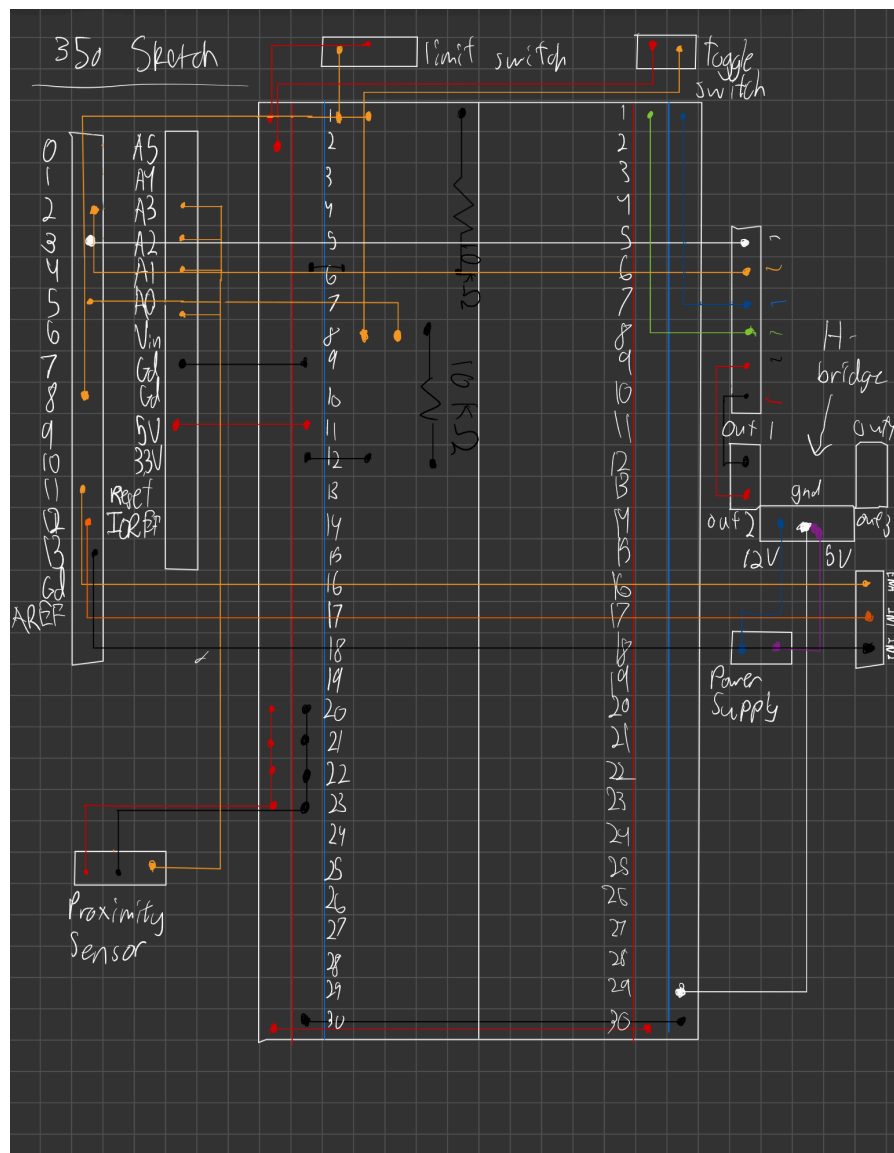


Figure D.1: Wiring Diagram

```
// ME350 Zombie Defense Sketch - Version 05.1 by Spencer Dickhudt
// updated 12-5-2024
//
//
////////////////////////////////////
// DEFINE CONSTANTS AND GLOBAL VARIABLES:    //
////////////////////////////////////

/** State Machine: **/

// CONSTANTS:

// Definition of states in the state machine
```

```

const int CALIBRATE = 1;
const int CHOOSE_ACTIVE_TARGET = 2;
const int MOVE_TO_TARGET = 3;

// VARIABLES:
// Global variable that keeps track of the state:
// Start the state machine in calibration state:
int state = CALIBRATE;

/** Proximity Sensors or Potentiometer: **/
// CONSTANTS:
// Definition of proximity sensor Limits for each target:
const int PROXIMITYSENSE1_MAX = 586; // [proximity sensor counts] Value of prox
sensor 1 when zombie is closest to photosensor
const int PROXIMITYSENSE1_MIN = 101; // [proximity sensor counts] Value of prox
sensor 1 when zombie is closest to prox sensor
const int PROXIMITYSENSE2_MAX = 573; // [proximity sensor counts] Value of prox
sensor 2 when zombie is closest to photosensor
const int PROXIMITYSENSE2_MIN = 142; // [proximity sensor counts] Value of prox
sensor 2 when zombie is closest to prox sensor
const int PROXIMITYSENSE3_MAX = 587; // [proximity sensor counts] Value of prox
sensor 3 when zombie is closest to photosensor
const int PROXIMITYSENSE3_MIN = 142; // [proximity sensor counts] Value of prox
sensor 3 when zombie is closest to prox sensor
const int PROXIMITYSENSE4_MAX = 590; // [proximity sensor counts] Value of prox
sensor 4 when zombie is closest to photosensor
const int PROXIMITYSENSE4_MIN = 95; // [proximity sensor counts] Value of prox
sensor 4 when zombie is closest to prox sensor

const int ProxRange[4][2] = { { PROXIMITYSENSE1_MAX, PROXIMITYSENSE1_MIN },
                              { PROXIMITYSENSE2_MAX, PROXIMITYSENSE2_MIN },
                              { PROXIMITYSENSE3_MAX, PROXIMITYSENSE3_MIN },
                              { PROXIMITYSENSE4_MAX, PROXIMITYSENSE4_MIN } }; //
Array holding prox sensor bounds to improve code readability

// Definition of target variables for array indexing
const int TARGET1 = 0; // For indexing into activeTargets array, activeTargets[0]
corresponds to Target 1
const int TARGET2 = 1; // For indexing into activeTargets array, activeTargets[1]
corresponds to Target 2

```

```

const int TARGET3 = 2; // For indexing into activeTargets array, activeTargets[2]
corresponds to Target 3
const int TARGET4 = 3; // For indexing into activeTargets array, activeTargets[3]
corresponds to Target 4

const int targetArr[4] = { TARGET1, TARGET2, TARGET3, TARGET4 }; // List of target
indices to reduce redundant code

// Direction variables to represent the movement of each zombie
const int FORWARD = 1; // representing the zombie moving towards the photosensor
const int BACKWARD = -1; // representing the zombie moving away from the photosensor
const int STOPPED = 0; // representing the zombie stalled

// VARIABLES:
// Variables for updating sensor readings
const float alpha = 0.925; // filter parameter used to weight current and past
readings
int stopTimeout = 250; // [millis] time needed to declare a zombie stopped
int noiseLimit = 8; // average spread of proximity readings for a
stopped zombie
const int lowerNoiseLimit = 5; // noise limit for near the photosensors
const int upperNoiseLimit = 8; // noise limit for near the prox sensors
const int noiseThreshold = 225; // prox sensor reading at which noise limit switches
from upper to lower

// A data structure to hold proximity sensor information:
struct Sensor {
    float currVal; // the most recent proximity sensor reading
    float prevVal; // the last update of proximity sensor reading
    unsigned long prevChangeTime; // [millis] the time at which prevVal last changed
    int pin; // the input pin of the desired prox sensor
    int direction; // the direction at which the zombie on the rail is
traveling
    int forwardCount; // number of readings which indicate the zombie is
moving forward
    int backwardCount; // number of readings which indicate the zombie is
moving backward
};

```

```

Sensor ProxSensors[4]; // An array which holds Sensor objects, representing each of
the prox sensors
int potValue = 0;      // Used to hold the reading of the potentiometer

int activeTargetIndex = -1; // index (in the activeTargets array) of the closest
target to the plants
int activeTargetPosition = -1; // [encoder counts] Encoder count of the active
target the linkage is moving toward

int minIndex = -1;      // tracking index of the zombie who is closest to the
plants
float closestZombieDist = 2; // [percentage] records the lowest percentage of rail
remaining for any of the zombies
int idx = -1;           // index of target, based on targetArr

bool WAIT_POS = true; // tracking if the linkage is moving to the wait position

unsigned long arrivalTime; // timer for tracking a wait period upon
reaching a desired position
const int targetActivateTime = 360; // time to activate a target
float Zombies[4]; // An array to hold information on locations of
all zombies in play

// float sampleTime = 1.5 // sample time in ms

/** Computation of position and velocity: **/
// CONSTANTS:
// Settings for velocity computation:
const int MIN_VEL_COMP_COUNT = 2; // [encoder counts] Minimal change in motor
position that must happen between two velocity measurements
const long MIN_VEL_COMP_TIME = 10000; // [microseconds] Minimal time that must pass
between two velocity measurements
// VARIABLES:
volatile int motorPosition = 0; // [encoder counts] Current motor position (Declared
'volatile', since it is updated in a function called by interrupts)
volatile int encoderStatus = 0; // [binary] Past and Current A&B values of the
encoder (Declared 'volatile', since it is updated in a function called by
interrupts)
// The rightmost two bits of encoderStatus will store the encoder values from the
current iteration (A and B).

```

```

// The two bits to the left of those will store the encoder values from the previous
iteration (A_old and B_old).
float motorVelocity = 0;           // [encoder counts / seconds] Current motor velocity
int previousMotorPosition = 0;     // [encoder counts] Motor position the last time a
velocity was computed
long previousVelCompTime = 0;     // [microseconds] System clock value the last time a
velocity was computed

/** High-level behavior of the controller: */
// CONSTANTS:
// Target positions:
const int CALIBRATION_VOLTAGE = -3.2; // [Volt] Motor voltage used during the
calibration process
const int TARGET_1_POSITION = 1305;   // [encoder counts] Motor position
corresponding to first target
const int TARGET_2_POSITION = 938;    // [encoder counts] Motor position
corresponding to second target
const int TARGET_3_POSITION = 645;    // [encoder counts] Motor position
corresponding to third target
const int TARGET_4_POSITION = 40;     // [encoder counts] Motor position
corresponding to fourth target
const int WAIT_POSITION = 900;        // [encoder counts] Motor position
corresponding to a wait position (when no targets are active)
const int LOWER_BOUND = 1322;        // [encoder counts] Position of the left end
stop
const int UPPER_BOUND = 0;            // [encoder counts] Position of the right end
stop
const int TARGET_BAND = 50;           // [encoder counts] "Close enough" range when
moving towards a target.

// List of target positions to reduce redundant code
const int targetPos[4] = { TARGET_1_POSITION, TARGET_2_POSITION, TARGET_3_POSITION,
TARGET_4_POSITION };

// Timing:
//const long WAIT_TIME = 0; // [microseconds] Time waiting for the target
to drop.
// TBD - implement a timer so if the target doesn't drop, the linkage moves to a
different target
// VARIABLES:

```

```
//unsigned long startWaitTime; // [microseconds] System clock value

/** PID Controller */
// CONSTANTS:
const float KP = 0.25;           // [Volt / encoder counts] P-Gain
const float KI = 0.001;          // [Volt / (encoder counts * seconds)] I-Gain
const float KD = 0.02;           // [Volt * seconds / encoder counts] D-Gain
const float SUPPLY_VOLTAGE = 9;  // [Volt] Supply voltage at the HBridge
const float FRICTION_COMP_VOLTAGE = 3; // [Volt] Voltage needed to overcome friction
// VARIABLES:
int desiredPosition = 0;          // [encoder counts] desired motor position
float positionError = 0;          // [encoder counts] Position error
float integralError = 0;          // [encoder counts * seconds] Integrated
position error
float velocityError = 0;          // [encoder counts / seconds] Velocity error
float desiredVoltage = 0;         // [Volt] Desired motor voltage
int motorCommand = 0;             // [0-255] PWM signal sent to the motor
unsigned long executionDuration = 0; // [microseconds] Time between this and the
previous loop execution. Variable used for integrals and derivatives
unsigned long lastExecutionTime = 0; // [microseconds] System clock value at the
moment the loop was started the last time

/** Pin assignment: */
// CONSTANTS:
const int PIN_NR_ENCODER_A = 2;    // Never change these, since the interrupts
are attached to pins 2 and 3
const int PIN_NR_ENCODER_B = 3;    // Never change these, since the interrupts
are attached to pins 2 and 3
const int PIN_NR_ON_OFF_SWITCH = 5; // Connected to toggle switch (turns
mechanism on and off)
const int PIN_NRL_LIMIT_SWITCH = 8; // Connected to limit switch (mechanism
calibration)
const int PIN_NR_PWM_OUTPUT = 11;   // Connected to H Bridge (controls motor
speed)
const int PIN_NR_PWM_DIRECTION_1 = 12; // Connected to H Bridge (controls motor
direction)
const int PIN_NR_PWM_DIRECTION_2 = 13; // Connected to H Bridge (controls motor
direction)
const int PIN_PROXSENSE1 = A0;       // Connected to proximity sensor 1
const int PIN_PROXSENSE2 = A1;       // Connected to proximity sensor 2
```

```

const int PIN_PROXSENSE3 = A2;          // Connected to proximity sensor 3
const int PIN_PROXSENSE4 = A3;          // Connected to proximity sensor 4
const int PIN_POTENTIOMETER = A4;       // Connected to potentiometer used to test
target positions

// Add this line of code if you want to use two limit switches; const int
PIN_NRL_LIMIT_SWITCH_2 = 11
// ^KEEP IN MIND THAT YOU HAVE TO ADD CODE DOWNSTREAM (FOR EXAMPLE YOU NEED TO ADD
THIS VARIABLE IN THE DECLARATION SECTION

// End of CONSTANTS AND GLOBAL VARIABLES

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/////
// The setup() function is called when a sketch starts. Use it to initialize
variables, //
// pin modes, start using libraries, etc. The setup function will only run once,
after //
// each powerup or reset of the Arduino board:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/////
void setup() {
    // Declare which digital pins are inputs and which are outputs:
    pinMode(PIN_NR_ENCODER_A, INPUT_PULLUP);
    pinMode(PIN_NR_ENCODER_B, INPUT_PULLUP);
    pinMode(PIN_NR_ON_OFF_SWITCH, INPUT);
    pinMode(PIN_NRL_LIMIT_SWITCH, INPUT);
    pinMode(PIN_PROXSENSE1, INPUT);
    pinMode(PIN_PROXSENSE2, INPUT);
    pinMode(PIN_PROXSENSE3, INPUT);
    pinMode(PIN_PROXSENSE4, INPUT);
    pinMode(PIN_NR_PWM_OUTPUT, OUTPUT);
    pinMode(PIN_NR_PWM_DIRECTION_1, OUTPUT);
    pinMode(PIN_NR_PWM_DIRECTION_2, OUTPUT);
    pinMode(PIN_POTENTIOMETER, INPUT);

    // Turn on the pullup resistors on the encoder channels

```

```

digitalWrite(PIN_NR_ENCODER_A, HIGH);
digitalWrite(PIN_NR_ENCODER_B, HIGH);

// Activate interrupt for encoder pins.
// If either of the two pins changes, the function 'updateMotorPosition' is called:
attachInterrupt(0, updateMotorPosition, CHANGE); // Interrupt 0 is always attached
to digital pin 2
attachInterrupt(1, updateMotorPosition, CHANGE); // Interrupt 1 is always attached
to digital pin 3

// Begin serial communication for monitoring.
Serial.begin(115200);
// Serial.println("Start Executing Program.");

// Initialize sensor values
ProxSensors[TARGET1].prevVal = analogRead(PIN_PROXSENSE1);
ProxSensors[TARGET2].prevVal = analogRead(PIN_PROXSENSE2);
ProxSensors[TARGET3].prevVal = analogRead(PIN_PROXSENSE3);
ProxSensors[TARGET4].prevVal = analogRead(PIN_PROXSENSE4);

// Initialize prox sensor pins
ProxSensors[TARGET1].pin = PIN_PROXSENSE1;
ProxSensors[TARGET2].pin = PIN_PROXSENSE2;
ProxSensors[TARGET3].pin = PIN_PROXSENSE3;
ProxSensors[TARGET4].pin = PIN_PROXSENSE4;

// Set initial output to the motor to 0
analogWrite(PIN_NR_PWM_OUTPUT, 0);
}
// End of function setup()

////////////////////////////////////
////////////////////////////////////
// After going through the setup() function, which initializes and sets the initial
values, //
// the loop() function does precisely what its name suggests, and loops
consecutively, //
// allowing your program to sense and respond. Use it to actively control the Arduino
board. //

```

```

////////////////////////////////////
////////////////////////////////////
void loop() {
    // Determine the duration it took to execute the last loop. This time is used
    // for integration and for monitoring the loop time via the serial monitor.
    executionDuration = micros() - lastExecutionTime;
    lastExecutionTime = micros();

    // Speed Computation:
    if ((abs(motorPosition - previousMotorPosition) > MIN_VEL_COMP_COUNT) || (micros()
- previousVelCompTime) > MIN_VEL_COMP_TIME) {
        // If at least a minimum time interval has elapsed or
        // the motor has travelled through at least a minimum angle ...
        // .. compute a new value for speed:
        // (speed = delta angle [encoder counts] divided by delta time [seconds])
        motorVelocity = (double)(motorPosition - previousMotorPosition) * 1000000 /
(micros() - previousVelCompTime);
        // Remember this encoder count and time for the next iteration:
        previousMotorPosition = motorPosition;
        previousVelCompTime = micros();
    }
    // ****
    // We first update the sensors so our direction information stays accurate

    // Loop through each of the sensors
    for (int i = 0; i < sizeof(targetArr) / sizeof(int); i++) {

        idx = targetArr[i];
        // Updates the current sensor reading using a low pass filter
        ProxSensors[idx].currVal = alpha * ProxSensors[idx].currVal + (1 - alpha) *
analogRead(ProxSensors[idx].pin);

        // Adjusts the noise Limit based on distance to sensor (since values are not
linear)
        if (ProxSensors[idx].currVal >= noiseThreshold) {
            noiseLimit = upperNoiseLimit;
        } else {
            noiseLimit = lowerNoiseLimit;
        }
    }
}

```

```

// Check if sensor is stopped
if (abs(ProxSensors[idx].currVal - ProxSensors[idx].prevVal) < noiseLimit) {
    // if not enough time has passed to declare stopped or already stopped continue
loop
    if (millis() - ProxSensors[idx].prevChangeTime < stopTimeout ||
ProxSensors[idx].direction == STOPPED) {
        ProxSensors[i].forwardCount = 0;
        ProxSensors[i].backwardCount = 0;
        continue;
    } else {
        // the prox sensor has been stopped for long enough that we know it is
stopped
        ProxSensors[idx].direction = STOPPED;
    }
} else if (ProxSensors[idx].currVal - ProxSensors[idx].prevVal < 0) {
    // Target is moving forward, as readings decrease moving further away from the
sensor
    ProxSensors[i].forwardCount += 1;
    // Target is moving forward
    if (ProxSensors[i].forwardCount > 3) {
        ProxSensors[i].direction = FORWARD;

        ProxSensors[i].prevVal = ProxSensors[i].currVal;
        ProxSensors[i].prevChangeTime = millis();
    }
} else {
    // Target must be moving backward
    ProxSensors[i].backwardCount += 1;
    // Target is moving forward
    if (ProxSensors[i].backwardCount > 3) {
        ProxSensors[i].direction = BACKWARD;

        ProxSensors[i].prevVal = ProxSensors[i].currVal;
        ProxSensors[i].prevChangeTime = millis();
    }
}
}
// *****//

//*****//

```

```

// The state machine:
switch (state) {
  //*****//
  // In the CALIBRATE state, we move the mechanism to a position outside of the
  // work space (towards the limit switch). Once the limit switch is on and
  // the motor has stopped turning, we know that we are against the end stop
  case CALIBRATE:
    // We don't have to do anything here since this state is only used to set
    // a fixed output voltage. This happens further below.

    // Decide what to do next:
    if (digitalRead(PIN_NRL_LIMIT_SWITCH) == HIGH && motorVelocity == 0) {
      // We reached the endstop. Update the motor position to the limit:
      // (NOTE: If the limit switch is on the right, this must be UPPER_BOUND)
      motorPosition = UPPER_BOUND;
      // Reset the error integrator:
      integralError = 0;
      // Calibration is finalized. Transition into DETERMINE_ACTIVE_TARGETS state
      // Serial.println("State transition from CALIBRATE to CHOOSE_ACTIVE_TARGET");
      state = CHOOSE_ACTIVE_TARGET;
    }

    // Otherwise we continue calibrating
    break;

  //*****//
  // In the DETERMINE_ACTIVE_TARGETS state, we use the most recent reading of the
  // photosensors to
  // calculate the location of each zombie.
  // We then select an active target to move toward based on which zombie is
  // closest to the plants.
  // We default to WAIT_POSITION if no targets are active.
  case CHOOSE_ACTIVE_TARGET:

    // Decide what zombie is closest to the photosensor
    // Current closest zombie is out of bounds
    minIndex = -1;
    // Set distance to 2 to be above any possible percentage values + noise
    closestZombieDist = 2;

```

```

    for (int i = 0; i < sizeof(targetArr) / sizeof(int); i++) {
        // Use the target array for the proper indexes into other arrays
        idx = targetArr[i];

        // Calculate the distance to the front of the rail by taking a percentage of
how much rail
        // the zombie has left to travel.
        Zombies[idx] = (ProxSensors[idx].currVal - ProxRange[idx][1]) /
(ProxRange[idx][0] - ProxRange[idx][1]);

        //Modify Zombie Priority to make sure that the shorter tracks are
prioritized--empirically found to increase score
        Zombies[0] = Zombies[0] * 0.99;
        Zombies[3] = Zombies[3] * 0.93;
        Zombies[1] = Zombies[1] * 0.85;
        Zombies[2] = Zombies[2] * 0.69;

        // Check to see if the zombie is traveling forward and it is closer than the
previous zombie
        if (ProxSensors[idx].direction == FORWARD && Zombies[idx] <
closestZombieDist) {
            // update the newest minimum value
            closestZombieDist = Zombies[idx];
            // update the index of the active target (in the targetArr array)
            minIndex = i;
        }
    }

    // if the min index points to a valid target, move to that target
    if (minIndex >= 0) {
        activeTargetIndex = targetArr[minIndex];
        activeTargetPosition = targetPos[activeTargetIndex];
        WAIT_POS = false;
        // Serial.println("Setting position to something other than wait");
    } else {
        activeTargetPosition = WAIT_POSITION;
        WAIT_POS = true;
        // Serial.println("Setting position to wait position");
    }
}

```

```

    state = MOVE_TO_TARGET;
    // Serial.println("Switching state to MOVE_TO_TARGET");

    // Otherwise, we stay in DETERMINE_ACTIVE_TARGETS
    break;

//*****//
// In the MOVE_TO_TARGET state, we select an active target and move toward it, or
// move toward Target 3 (a default position) if there is no active target
case MOVE_TO_TARGET:
    // Serial.println("Inside MOVE_TO_TARGET");
    desiredPosition = activeTargetPosition;

    if (motorPosition <= activeTargetPosition + TARGET_BAND && motorPosition >=
activeTargetPosition - TARGET_BAND) {
        if (desiredPosition == TARGET_4_POSITION) {
            if (millis() - arrivalTime > (targetActivateTime) || WAIT_POS) {
                state = CHOOSE_ACTIVE_TARGET;
                // Serial.println("PICKING TARGET");
            }
        } else {
            if (millis() - arrivalTime > targetActivateTime || WAIT_POS) {
                state = CHOOSE_ACTIVE_TARGET;
                // Serial.println("PICKING TARGET");
            }
        }
    } else {
        arrivalTime = millis();
    }

    break;

//*****//
//*****//
// We should never reach the next bit of code, which would mean that the state
// we are currently in doesn't exist. So if it happens, throw an error and
// stop the program:
default:

```

```

    // Serial.println("Statemachine reached at state that it cannot handle.
ABORT!!!!");
    // Serial.print("Found the following unknown state: ");
    // Serial.println(state);
    while (1)
        ; // infinite loop to halt the program
    break;
}
// End of the state machine.
//*****//

//*****//
// Recalibrate if we are in the leftmost position
if (digitalRead(PIN_NRL_LIMIT_SWITCH) == HIGH && motorVelocity == 0) {
    // We reached the endstop. Update the motor position to the limit:
    // (NOTE: If the limit switch is on the right, this must be UPPER_BOUND)
    motorPosition = UPPER_BOUND;
    // Reset the error integrator:
    integralError = 0;
    // Serial.println("Limit Switch hit");
}

//*****//
// Position Controller
if (digitalRead(PIN_NR_ON_OFF_SWITCH) == HIGH) {
    // If the toggle switch is on, run the controller:

    /** PID control: **/
    // Compute the position error [encoder counts]
    positionError = desiredPosition - motorPosition;
    // Compute the integral of the position error [encoder counts * seconds]
    integralError = integralError + positionError * (float)(executionDuration) /
1000000;
    // Compute the velocity error (desired velocity is 0) [encoder counts / seconds]
    velocityError = 0 - motorVelocity;
    // This is the actual controller function that uses the error in
    // position and velocity and the integrated error and computes a
    // desired voltage that should be sent to the motor:
    desiredVoltage = KP * positionError + KI * integralError + KD * velocityError;

```

```

/** Feedforward terms: **/
// Compensate for friction. That is, if we know the direction of
// desired motion, add a base command that helps with moving in this
// direction:
if (positionError < -5) {
    desiredVoltage = desiredVoltage - FRICTION_COMP_VOLTAGE;
}
if (positionError > +5) {
    desiredVoltage = desiredVoltage + FRICTION_COMP_VOLTAGE;
}

// Anti-Wind-Up
if (abs(desiredVoltage) > SUPPLY_VOLTAGE) {
    // If we are already saturating our output voltage, it does not make
    // sense to keep integrating the error (and thus ask for even higher
    // and higher output voltages). Instead, stop the integrator if the
    // output saturates. We do this by reversing the summation at the
    // beginning of this function block:
    integralError = integralError - positionError * (float)(executionDuration) /
1000000;
}
// End of 'if(onOffSwitch==HIGH)'

// Override the computed voltage during calibration. In this state, we simply
apply a
// fixed voltage to move against one of the end-stops.
if (state == CALIBRATE) {
    // add calibration code here
    desiredVoltage = CALIBRATION_VOLTAGE;
}
} else {
    // Otherwise, the toggle switch is off, so do not run the controller,
    // stop the motor...
    desiredVoltage = 0;
    // .. and reset the integrator of the error:
    integralError = 0;
    // Produce some debugging output:
    Serial.println("The toggle switch is off. Motor Stopped.");
}

```

```

// End of else onOffSwitch==HIGH

/** Send signal to motor */
// Convert from voltage to PWM cycle:
motorCommand = int(abs(desiredVoltage * 255 / SUPPLY_VOLTAGE));
// Clip values larger than 255
if (motorCommand > 255) {
    motorCommand = 255;
}
// Send motor signals out
analogWrite(PIN_NR_PWM_OUTPUT, motorCommand);
// Determine rotation direction
if (desiredVoltage >= 0) {
    // If voltage is positive ...
    // ... turn forward
    digitalWrite(PIN_NR_PWM_DIRECTION_1, LOW); // rotate forward
    digitalWrite(PIN_NR_PWM_DIRECTION_2, HIGH); // rotate forward
} else {
    // ... otherwise turn backward:
    digitalWrite(PIN_NR_PWM_DIRECTION_1, HIGH); // rotate backward
    digitalWrite(PIN_NR_PWM_DIRECTION_2, LOW); // rotate backward
}
// End of Position Controller
//*****//

// Print out current controller state to Serial Monitor.
printStatsToSerial();
}
// End of main loop
//*****//

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// This is a function to update the encoder count in the Arduino. //
// It is called via an interrupt whenever the value on encoder //
// channel A or B changes. //
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
void updateMotorPosition() {
    // Bitwise shift left by one bit, to make room for a bit of new data:
    encoderStatus <<= 1;

```

```

    // Use a compound bitwise OR operator (|=) to read the A channel of the encoder
(pin 2)
    // and put that value into the rightmost bit of encoderStatus:
    encoderStatus |= digitalRead(2);
    // Bitwise shift left by one bit, to make room for a bit of new data:
    encoderStatus <<= 1;
    // Use a compound bitwise OR operator (|=) to read the B channel of the encoder
(pin 3)
    // and put that value into the rightmost bit of encoderStatus:
    encoderStatus |= digitalRead(3);
    // encoderStatus is truncated to only contain the rightmost 4 bits by using a
    // bitwise AND operator on mstatus and 15(=1111):
    encoderStatus &= 15;
    if (encoderStatus == 2 || encoderStatus == 4 || encoderStatus == 11 ||
encoderStatus == 13) {
        // the encoder status matches a bit pattern that requires counting up by one
        motorPosition++; // increase the encoder count by one
    } else if (encoderStatus == 1 || encoderStatus == 7 || encoderStatus == 8 ||
encoderStatus == 14) {
        // the encoder status does not match a bit pattern that requires counting up by
one.
        // Since this function is only called if something has changed, we have to count
downwards
        motorPosition--; // decrease the encoder count by one
    }
}
// End of function updateMotorPosition()

////////////////////////////////////
// This function sends a status of the controller to the serial    //
// monitor. Each character will take 85 microseconds to send, so  //
// be selective in what you write out:                               //
////////////////////////////////////
void printStateToSerial() {
    //*****//
    // Send a status of the controller to the serial monitor.
    // Each character will take 85 microseconds to send, so be selective
    // in what you write out:

```

```

//Serial.print("State Number:  [CALIBRATE = 1; DETERMINE_ACTIVE_TARGETS = 2;
MOVE_TO_TARGET = 3]: ");
// Serial.print("State#: ");
// Serial.print(state);

//Serial.print("Power switch [on/off]: ");
//Serial.print("  PWR: ");
//Serial.print(digitalRead(PIN_NR_ON_OFF_SWITCH));

//Serial.print("      Motor Position [encoder counts]: ");
// Serial.print("  MP: ");
// Serial.print(motorPosition);

//Serial.print("      Motor Velocity [encoder counts / seconds]: ");
// Serial.print("  MV: ");
// Serial.print(motorVelocity);

//Serial.print("      Encoder Status [4 bit value]: ");
//Serial.print("  ES: ");
//Serial.print(encoderStatus);

// //Serial.print("      Target Position [encoder counts]: ");
// Serial.print("  DP: ");
// Serial.print(desiredPosition);

// //Serial.print("      Position Error [encoder counts]: ");
// Serial.print("  PE: ");
// Serial.print(positionError);

// //Serial.print("      Integrated Error [encoder counts * seconds]: ");
// Serial.print("  IE: ");
// Serial.print(integralError);

// //Serial.print("      Velocity Error [encoder counts / seconds]: ");
// Serial.print("  VE: ");
// Serial.print(velocityError);

// //Serial.print("      Desired Output Voltage [Volt]: ");
// Serial.print("  DV: ");
// Serial.print(desiredVoltage);

```

```

//Serial.print("      Motor Command [0-255]: ");
//Serial.print("  MC: ");
//Serial.print(motorCommand);

//Serial.print("      Execution Duration [microseconds]: ");
//Serial.print("  ED: ");
//Serial.print(executionDuration);

//Serial.print("      Zombie Location [% of rail left to travel]: ");
// Serial.print("  ZL: ");
// for (int i = 0; i < sizeof(Zombies)/sizeof(float); i++) {
//   Serial.print(Zombies[i]);
//   Serial.print(" ");
// }

//Serial.print("      Proxsensor Direction[STOPPED or FORWARD or BACKWARD]: ");
// Serial.print("  DR: ");
// for (int i = 0; i < sizeof(Zombies)/sizeof(float); i++) {
//   Serial.print(ProxSensors[i].direction);
//   Serial.print(" ");
// }

// // ALWAYS END WITH A NEWLINE.  SERIAL MONITOR WILL CRASH IF NOT
// Serial.println(); // new line
}
// End of Serial Out

////////////////////////////////////
// This function returns the average of the integer array          //
// pointed to by array_ptr and of length len.                      //
////////////////////////////////////
float average(int* array_ptr, int len) {
  long sum = 0L;
  for (int i = 0; i < len; i++) {
    sum += array_ptr[i];
  }
  return ((float)sum) / len;
}
// End of average

```

